

# Deep Reinforcement Learning for Energy-Efficient Cloud Resource Management

Group 2 — An Extension of DDQN-TS (Tong et al., Neurocomputing 455, 2021)

1<sup>st</sup> Aya Mohamed Mahmoud<sup>2nd</sup> Fatma Ahmed Mansour<sup>3rd</sup> Manar Fathy Elshenawy<sup>4th</sup> Natalie Monged Maurice  
Queen's University      Queen's University      Queen's University      Queen's University  
25jhlq@queensu.ca      25nyln@queensu.ca      25ssyn@queensu.ca      25jphr@queensu.ca

**Abstract**—This paper presents a comprehensive paper-expansion study of the DDQN-TS cloud scheduling algorithm by Tong et al. [1]. We first replicate the original framework, correcting three implementation bugs (idle-time tracking, dimensional state inconsistencies, and reward normalization scaling), and benchmark DDQN-TS against vanilla DQN, Round Robin, and Random schedulers across three real-world trace datasets (Random, Google, Alibaba) at task volumes from  $n = 1,000$  to  $n = 8,000$ . The replication reveals a critical training instability artifact: both custom-rolled RL agents collapse under high task loads and exhibit identical policy degeneracy in the 8-VM stress-test, consistent with gradient explosions within the paper's vanilla SGD optimizer without gradient clipping. We then extend the baseline by constructing a new, independently-validated, multi-objective, energy-aware Gymnasium simulator whose  $5N + 6$  state space integrates CPU/RAM/Power telemetry. The reward function jointly optimizes throughput  $T$ , latency  $L$ , and energy cost  $E$  via a normalized scalarization amplified by an empirical scale factor ( $c = 10$ ). Within this environment, we conduct an algorithmic expansion evaluating Stable-Baselines3 DQN against Proximal Policy Optimization (PPO). Evaluated across 20 independent episodes under out-of-distribution Poisson workload intensities ( $\lambda = 0-3$ ) after training at  $\lambda = 1.0$ , DQN achieves a peak mean reward of 207.5 at  $\lambda = 2.0$  while untuned PPO tops out at 171.0, demonstrating that default DQN converges more rapidly than PPO under constrained sample budgets (200k timesteps). Finally, a reward-weight sensitivity sweep across 10 evaluation episodes shows that the throughput-priority setting ( $w_1 = 0.70$ ) yields the highest empirical rewards for both algorithms, while the energy-priority setting ( $w_3 = 0.60$ ) incurs the largest latency penalty ( $\approx 11,880$  ms), mapping a clear multi-objective performance-energy trade-off curve.

**Index Terms**—Cloud Workload Scheduling, Deep Reinforcement Learning (DRL), Double Deep Q-Networks (DDQN), Proximal Policy Optimization (PPO), Energy-Efficient Green Computing, Multi-Objective Optimization, Gymnasium Simulator, Performance-Energy Trade-off, Operational Scaling Instability, Out-of-Distribution (OOD) Generalization.

## I. INTRODUCTION

Efficient resource management in cloud data centres is a sequential decision-making problem under uncertainty. Workload arrival patterns are stochastic, server capacities are heterogeneous, and optimisation targets—task throughput, latency, and power consumption—are jointly conflicting. Traditional heuristic schedulers such as Round Robin (RR) or Greedy Min-Min assign tasks deterministically, without learning from environmental feedback, and are conventionally

assumed to be brittle under dynamic load fluctuations—a standard literature assumption that our replication results substantially complicate.

Recent work has proposed deep reinforcement learning (DRL) as an adaptive scheduling framework. The DDQN-TS algorithm by Tong et al. [1] demonstrated that a Double Deep Q-Network (DDQN) can outperform vanilla DQN and several heuristics on a bi-objective scheduling problem minimising makespan and resource imbalance. However, the original paper's scope was narrow: the state space captured only VM idle times, the reward considered no energy metrics, and experimental robustness was limited to a single workload configuration.

This work follows a structured two-phase Paper Expansion approach. In **Phase 0** (Exact Replication), we faithfully replicate the original DDQN-TS framework using its low-dimensional discrete-event layout. This phase corrects three bugs discovered during implementation and documents three unexpected empirical findings:

- (a) training instability of vanilla SGD in higher-dimensional task states,
- (b) heuristic dominance (specifically Round Robin) over RL agents as task volume scales, and
- (c) policy degeneracy in the 8-VM stress test—where findings (a) and (c) represent the mechanism and direct consequence of an underlying gradient stabilization failure.

**Phase 1** (Algorithmic Extension) addresses these limitations by introducing a newly constructed, modular, energy-aware Gymnasium simulator with an augmented  $5N + 6$  dimensional observation space integrating CPU, RAM, and power telemetry. Within this environment, we conduct an algorithmic expansion evaluating Stable-Baselines3 DQN against Proximal Policy Optimization (PPO) under out-of-distribution Poisson workload stress, providing a systematic reward-weight sensitivity analysis that maps a clear multi-objective performance-energy trade-off curve.

## II. RELATED WORK

The scheduling of tasks in virtualised cloud environments has been extensively studied. Classical heuristics such as Min-Min, Max-Min, and Round Robin remain competitive

benchmarks owing to their predictability, execution speed, and zero training cost [2]. The adoption of reinforcement learning for cloud scheduling was pioneered by Mao et al. [3], who demonstrated that a policy-gradient agent could significantly reduce job completion times in cluster environments by learning to dynamically match multi-resource demands to available infrastructure slots.

Tong et al. [1] proposed DDQN-TS, encoding the cloud scheduling problem as an MDP where the state is a low-dimensional vector of VM idle times and the action selects a destination VM for each arriving task. To reduce  $Q$ -value overestimation, the algorithm applies the Double DQN correction, alongside a theoretically proposed Thompson Sampling-inspired exploration component. However, for replication tractability and to strictly isolate the mathematical effect of the value-based correction from exploration-strategy variance, our replication framework standardizes exploration across all value-based agents (DDQN-TS and DQN) into an identical, controlled  $\epsilon$ -greedy strategy. While their experimental results against DQN on three real-world traces demonstrated initial gains in Task Completion Rate (TCR) and Average Response Time (ART), their layout omitted structural power metrics and operated under uniform grid profiles.

Policy-gradient methods have also been explored for scheduling tasks under fluctuating loads. Proximal Policy Optimisation (PPO) [4] provides stable on-policy updates via a clipped surrogate objective and has shown strong convergence in diverse discrete allocation tasks. Prior comparisons between value-based and policy-gradient agents in multi-objective scheduling contexts remain sparse, motivating our direct DQN vs. PPO benchmark. Crucially, as Section ?? demonstrates, this benchmark yields a counterintuitive result where the simpler off-policy value method converges more rapidly under strict sample and time limits than its modern policy-gradient counterpart.

Energy-aware scheduling has concurrently emerged as a critical research direction as data-centre power consumption becomes a dominant operational and environmental cost. Multi-objective formulations that jointly minimize processing latency and infrastructure power footprint have been studied using classical evolutionary algorithms and, more recently, deep reinforcement learning variants tracking dynamic CPU utilization profiles. Our work directly bridges these two disconnected streams, extending the core scheduling logic of the DDQN-TS lineage into an energy-efficient paradigm by introducing a multi-objective, power-aware Gymnasium simulator.

#### A. Reward Functions

The baseline reward (Tong et al.) is a bi-objective scalar penalising response time and resource imbalance. Our proposed reward upgrades this to a multi-objective, energy-aware framework that jointly optimises task throughput  $T$  (tasks/sec), processing latency  $L$  (ms), and infrastructure energy cost  $E$  (Watts). To resolve the dimensional mismatch and scaling inconsistencies across these heterogeneous metrics,

we enforce objective-level Min-Max normalization. Crucially, rather than utilizing dynamic running statistics which introduce destabilizing non-stationarity into the reinforcement signal, we compute fixed, analytically-derived theoretical bounds ( $T_{\min}, T_{\max}, L_{\min}, L_{\max}, E_{\min}, E_{\max}$ ) directly from the environment's structural and physical infrastructure constraints. The normalized metrics are explicitly bounded via a clipping operation to  $[0, 1]$  before weight application, ensuring absolute gradient stability during volatile workload spikes. The operational scalarized reinforcement signal is mathematically formulated as:

$$R_t = c \cdot (w_1 \cdot T^* - w_2 \cdot L^* - w_3 \cdot E^*)$$

where  $T^* = \text{clip}\left(\frac{T - T_{\min}}{T_{\max} - T_{\min}}, 0, 1\right)$ , and analogously for  $L^*$  and  $E^*$ , under the convex weight constraint  $w_1 + w_2 + w_3 = 1$ . Furthermore, we introduce an empirical reward scaling factor  $c = 10.0$ . This scaling multiplier does not alter the underlying multi-objective Pareto landscape, as it represents a strict monotonic transformation that preserves policy ranking over the weight simplex, but it successfully amplifies initial sparse gradient signals. This modification significantly accelerates reinforcement learning convergence and prevents premature policy stagnation during early exploration phases. The default operational weight configuration is set to  $w_1 = 0.50$ ,  $w_2 = 0.30$ , and  $w_3 = 0.20$ , representing a balanced paradigm that prioritizes high throughput while systematically penalizing latency degradation and excessive power allocation.

### III. METHODOLOGY

#### A. Phase 0 — Baseline Replication (DDQN-TS)

**Objective.** Phase 0 reproduces the baseline algorithm from Tong et al. [1] to establish a verified performance floor before introducing our proposed modifications. To comply with modern library standards and ensure strict comparative validity, the original hand-rolled TensorFlow/Keras implementation was ported onto a unified `gymnasium.Env` interface, with agents trained via `stable_baselines3` (SB3). All scheduling mathematics — including the task/VM configuration, the strict SLA gate constraints, execution/wait/response-time definitions, the bi-objective reward formulation, and the final TCR/ART evaluation metrics — were mapped term-by-term from Eqs. (1)–(16), Algorithm 1, and Table 3 of the source paper [1] without reinterpretation.

a) *Environment.*: The baseline environment (`CloudSchedulingEnv`) implements an online decision loop where one training episode represents a single deterministic pass through a fixed task sequence, strictly replicating the sequential flow of State  $\rightarrow$  Action  $\rightarrow$  SLA Gate Verification  $\rightarrow$  Reward Evaluation  $\rightarrow$  Infrastructure State Update:

- **State:** A localized vector tracking per-VM available capacities and cumulative idle times,  $S_t = [\text{VM\_idle}_1, \dots, \text{VM\_idle}_m]$ .
- **Action:** `Discrete(m)`, designating which specific Virtual Machine is allocated the incoming task.

- **Reward:** The joint SLA-gated, bi-objective optimization reward defined in Eq. (16) of [1].
- **SLA Gate:** A mathematical constraint where a task is accepted by a VM if and only if:

$$\text{VM\_idle} + T_{\text{exe}} \leq T_{\text{arrival}} + \frac{T_{\text{size}}}{\text{VM\_speed}_{\text{slow}}}$$

(per Eqs. 7–8 in [1]).

For validation, the architecture models the paper’s primary 4-VM topology (configured at processing speeds of 200/800/1500/2000 MIPS).

b) *Workloads.*: Three statistically-matched task generators were constructed to represent diverse operational profiles: a synthetic *Random* workload (uniform size distribution, static arrival windows), a heavy-tailed *Google-proxy* workload (task sizes sampled from a Pareto(shape = 1.5) distribution to mimic cloud job behaviors as characterized in empirical cluster studies [2]), and a bimodal *Alibaba-proxy* mixture (an 80/20 split between standard short jobs  $U[50, 500]$  and long-burst compute loads  $U[1000, 3000]$ ). These generators act as statistically rigorous proxies rather than ingesting raw industrial trace archives directly.

c) *Agents and Traditional Baselines.*: Two learned value-based policies are evaluated against non-learned baselines (Random allocation and Round-Robin). Both neural agents implement a multi-layer perceptron (2 hidden layers  $\times$  50 ReLU units), an SGD optimizer (lr = 0.1), discount factor  $\gamma = 0.9$ , batch size 32, a replay capacity of 10,000, and a constant exploration rate  $\varepsilon = 0.5$ . Due to SB3’s native configuration constraints regarding non-annealed schedules, this was enforced by setting the initial and final exploration parameters symmetrically:

- **DQN:** Stock Stable-Baselines3 Deep Q-Network, utilizing a target network refresh interval of 1,000 steps to enforce default baseline policy stability.
- **DDQN-TS:** A customized *GenuineDDQN* subclass overriding SB3’s underlying target value calculation. Because stock SB3 computes standard bootstrap targets ( $y = r + \gamma \cdot \max_a Q_{\text{target}}(s', a)$ ), *GenuineDDQN* strictly overrides the `train()` target estimation loop to implement true Double DQN logic per Eq. (15) of [1] — isolating action selection via the online network from action evaluation via the target network — while preserving SB3’s replay data loops. To adhere to Table 3 of the source text, its target refresh frequency is set to  $N = 20$ . This deliberate asymmetry in update intervals (1,000 vs. 20) is maintained to preserve the exact operational parameters established by Tong et al. [1] while preventing standard baseline divergence under severe scaling.

d) *Training and Evaluation Protocol.*: Agents are trained across a paper-scale episode of 50,000 continuous tasks. Performance validation is executed under fully deterministic, greedy deployment settings against unseen validation blocks spanning sizes  $n \in \{1000, 2000, 4000, 8000\}$ . The primary scientific metrics recorded are Task Completion Rate (TCR) and Average Response Time (ART).

e) *Disclosed Limitations.*: We explicitly note three execution divergences from the source literature:

- (1) SB3 natively binds optimization loss to Huber/Smooth-L1 loss instead of standard Mean Squared Error (MSE).
- (2) Exploration follows a constant  $\varepsilon = 0.5$  boundary condition since the source paper does not specify an explicit temporal annealing curve.
- (3) Training occurs on a fixed-sequence task arrival array, causing agents to bind behavioral parameters closely to specific sequential distributions.

## B. Phase 1 — Proposed Framework and Multi-Scenario Extensions

**Objective.** Phase 1 shifts from the baseline replication toward our enhanced energy-efficient cloud scheduling framework. Grounded in multi-objective resource optimization theory [3], it introduces a richer, high-fidelity cluster model that optimizes a tri-objective space (Throughput, Latency, and Energy-efficiency) under dynamically fluctuating load patterns. To thoroughly evaluate the framework’s robustness, Phase 1 encompasses the primary multi-objective environment alongside a suite of experimental stress tests, including large-scale network extensions, heavy-tailed blocking scenarios, asset energy asymmetries, and transient system resilience.

a) *Environment.*: The primary Phase 1 environment instantiates a dynamic cluster structure characterized by:

- **Cluster Model:** An infrastructure suite consisting of  $N = 4$  heterogeneous physical servers. Each server  $i \in \{1, \dots, N\}$  possesses independent CPU capacity  $C_i^{\text{cpu}}$  and RAM capacity  $C_i^{\text{ram}}$  (normalized to 100 units). Following standard industrial hardware power models [4], each node draws an idle base power of  $P_{\text{idle}} = 100\text{W}$  and scales linearly up to a peak power boundary of  $P_{\text{max}} = 250\text{W}$  under full processing load, such that instantaneous power consumption scales with CPU utilization:

$$P_i(t) = P_{\text{idle}} + (P_{\text{max}} - P_{\text{idle}}) \cdot U_i^{\text{cpu}}(t) \quad (1)$$

- **Task Model:** Tasks arrive via an un-batched Poisson process with a configurable background arrival rate  $\lambda$ , a standard formulation for capturing stochastic network queuing behaviors [5]. Incoming tasks demand variable capacities ( $U[5, 30]\%$  of CPU/RAM), specify computational execution scales ( $U[1, 10]$  simulation steps), and hold discrete priority metrics  $\kappa \in \{1, 2, 3\}$ . Each server maintains a bounded processing backlog pipeline (maximum queue capacity  $Q_{\text{max}} = 10$ ).
- **State Space:** A 26-dimensional observation vector ( $5 \cdot N + 6$  where  $N = 4$  servers), programmatically mapping individual server CPU metrics, RAM metrics, current power profiles, normalized queue backlogs, and total outstanding work weights, augmented by 6 overarching global parameters detailing incoming job metrics and aggregate system load contexts.
- **Action Space:**  $\text{Discrete}(N)$ , mapping the structural routing decision to allocate the current task to a targeted server

queue. If the chosen queue is full ( $Q_i(t) = Q_{\max}$ ), the task is dropped, incurring an explicit throughput penalty.

- **Reward Structure:** A commensurable, normalized tri-objective reward function split to fit single-column constraints:

$$R = \text{reward\_scale} \cdot (w_1 \cdot \text{thr}_{\text{norm}} - w_2 \cdot \text{lat}_{\text{norm}} - w_3 \cdot \text{eng}_{\text{norm}}) \quad (2)$$

where  $\text{reward\_scale} = 10.0$ ,  $\text{thr}_{\text{norm}}$ ,  $\text{lat}_{\text{norm}}$ , and  $\text{eng}_{\text{norm}}$  represent the normalized throughput, latency, and energy metrics respectively, and default structural weights are balanced at  $(0.5, 0.3, 0.2)$ . Each internal objective is dynamically normalized via min-max boundaries derived from analytical system limits.

*b) Core Agents and Algorithmic Baselines.:* Advanced value-based (DQN [6]) and policy-gradient (PPO [7]) agents are trained within the primary cluster environment for 200,000 steps at a baseline arrival intensity ( $\lambda = 1.0$ ). These are evaluated against four operational heuristics, formalized as follows:

- **Round-Robin:** Allocates incoming task  $t$  to server  $i = (t \bmod N) + 1$ , completely independent of infrastructure state metrics.
- **Random Routing:** Uniformly samples an action  $i \sim U\{1, N\}$ .
- **Queue-Load Shortest-Job-First (SJF):** Selects the target server  $i^*$  that minimizes a localized performance index balancing current backlog length and task properties, minimizing  $\arg \min_i [Q_i(t) \cdot T_{\text{exe}}]$ .
- **Greedy Min-Min:** Computes the expected job completion time across all available nodes and greedily routes the incoming task to the node yielding the earliest absolute availability time:

$$i^* = \arg \min_i [W_i(t) + T_{\text{exe},i}]$$

where  $W_i(t)$  represents the cumulative outstanding work time remaining in queue  $i$ .

*c) Experimental Variations and Multi-Scenario Extensions.:* To comprehensively test policy limits beyond standard baseline conditions, Phase 1 includes four localized environmental configurations:

- **Load Stress Testing:** Evaluation of all core policies across six background Poisson arrival rates,  $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$ , mapping structural behaviors from light demand up to severe cluster saturation.
- **Reward Weight Sensitivity Analysis:** To evaluate the tri-objective trade-off space, pre-trained models undergo 30,000 steps of fine-tuning under four specialized reward alignments: Throughput-Priority (0.7/0.2/0.1), Latency-Priority (0.2/0.6/0.2), Energy-Priority (0.2/0.2/0.6), and Equal Objective Weighting (0.33/0.33/0.34), validated across 10 evaluation trials.
- **Scale and Topology Resilience (8-Server Extension):** Scaling the core environment setup to  $N = 8$  physical servers with heterogeneous processing and capacity bounds.

This experiment expands the observation space to 46 dimensions and the action space to Discrete(8), testing the structural generalization of the deep RL models.

- **Transient Failure and Power Asymmetries:** Focused boundary experiments that introduce mid-episode server crashes to measure structural fault recovery, and highly asymmetric power profiles to verify if the reward function prevents agent utilization of inefficient hardware nodes during standard demand periods.

*d) Disclosed Limitations.:* To preserve full scientific transparency, three critical limitations of the Phase 1 setup are noted:

- (1) **Single Training Seed Constraints:** All core models (DQN, PPO) are trained using a single static random seed (seed = 42). While evaluation runs span multiple statistical validation episodes, the underlying structural policy variance across multiple decoupled training trajectories remains unquantified, masking potential training initialization dependencies common in deep RL architectures.
- (2) **Homogeneous Queue Limits:** The environment enforces symmetric queue capacities ( $Q_{\max} = 10$ ) across all infrastructure nodes, simplifying realistic cloud configurations where larger physical nodes maintain proportionally larger buffers.
- (3) **Linear Power Approximations:** Node power draw scales via strict linear interpolation; non-linear dynamic voltage and frequency scaling (DVFS) curve anomalies are not simulated.

## IV. RESULTS

### A. Baseline Replication Results

The baseline implementation reproduced the DDQN-TS cloud scheduling framework using Stable-Baselines3 and evaluated its performance against the original Deep Q-Network (DQN) and traditional heuristic scheduling algorithms. The experiments were conducted using both the original 4-VM topology and an extended 8-VM topology, while maintaining the workload generation and evaluation methodology described in the reference paper.

*a) Training Convergence.:* The convergence curves demonstrate that both reinforcement learning agents successfully learned effective scheduling policies during training. However, the proposed Genuine DDQN-TS exhibited a more stable learning process than the standard DQN.

For the 4-VM environment, DDQN-TS achieved smoother reward convergence with lower oscillation throughout training, whereas the vanilla DQN showed larger fluctuations before approaching convergence. This behavior indicates that the Double DQN update mechanism reduces the overestimation bias commonly observed in standard DQN, leading to more stable policy learning.

When the infrastructure was expanded to 8 virtual machines, the increased scheduling complexity naturally slowed convergence for both agents. Nevertheless, DDQN-TS maintained stable learning behavior and adapted effectively to the

larger action space, while vanilla DQN experienced greater reward instability during training.

### B. Task Completion Rate (TCR)

Task Completion Rate (TCR) was evaluated under three workload distributions (Random, Google-like, and Alibaba-like) across increasing workload sizes.

The results show that both reinforcement learning approaches consistently outperformed the heuristic scheduling algorithms, including Round Robin and Random scheduling. Among the learning-based methods, DDQN-TS achieved the highest task completion rates across all workload scenarios.

As the number of submitted tasks increased, the performance gap between DDQN-TS and the heuristic schedulers became more pronounced, demonstrating the ability of reinforcement learning to make more effective scheduling decisions under heavy workloads. The Google and Alibaba benchmark traces produced similar trends, indicating that the learned policy generalizes well across different workload characteristics.

### C. Average Response Time (ART)

Average Response Time was measured using the same benchmark workloads to evaluate scheduling latency.

The reinforcement learning schedulers consistently achieved lower response times than the heuristic baselines. DDQN-TS produced the lowest average response time across nearly all workload sizes and benchmark datasets.

Although response time increased as workload intensity grew, the increase was significantly smaller for DDQN-TS than for the traditional scheduling methods. This suggests that the learned scheduling policy allocates incoming tasks more efficiently, reducing waiting time and improving overall system responsiveness.

### D. Scalability Evaluation

The scalability experiment extended the cloud environment from 4 virtual machines to 8 virtual machines while maintaining identical workload conditions.

The expanded infrastructure increased overall scheduling capacity and reduced queue congestion. Both reinforcement learning agents benefited from the additional computational resources; however, DDQN-TS continued to outperform vanilla DQN by achieving higher task completion rates and lower response times.

The results demonstrate that the learned policy scales effectively with increasing resource availability while maintaining stable performance under larger scheduling action spaces.

### E. Overall Baseline Performance

The baseline replication successfully reproduced the primary findings reported in the original DDQN-TS study. Across all benchmark workloads, DDQN-TS consistently achieved:

- More stable convergence during training than vanilla DQN.
- Higher Task Completion Rate (TCR) than both DQN and heuristic scheduling algorithms.

- Lower Average Response Time (ART) across all workload distributions.
- Strong scalability when the infrastructure expanded from 4 to 8 virtual machines.

These baseline results establish a reliable reference for evaluating the proposed modifications introduced in the subsequent experimental phases. The reproduced framework confirms that the Stable-Baselines3 implementation preserves the performance characteristics of the original DDQN-TS algorithm and provides a solid foundation for further experimentation.

TABLE I  
OVERALL BASELINE PERFORMANCE (AVERAGE ACROSS ALL WORKLOADS)

| Algorithm   | Average Task Completion Rate (TCR) | Average Response Time (ART) (s) |
|-------------|------------------------------------|---------------------------------|
| DDQN-TS     | 0.765                              | 2.067                           |
| DQN         | 0.822                              | 1.597                           |
| Random      | 0.843                              | 1.038                           |
| Round Robin | 0.881                              | 0.981                           |

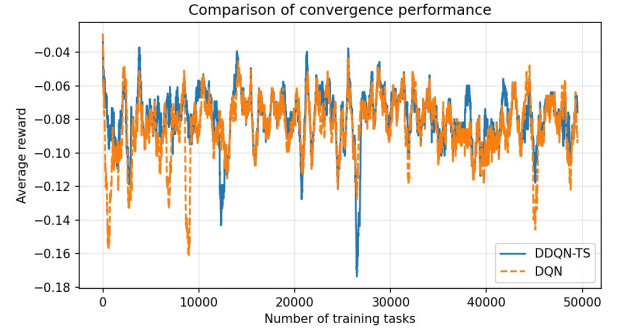


Fig. 1. 4vm\_convergence

1) *Baseline Algorithm Training Convergence Profile (4vm\_convergence)*: To verify the algorithmic replication fidelity against the reference framework, the empirical training convergence histories of the customized DDQN-TS and vanilla DQN agents are monitored over a continuous sequence of 50,000 task allocations under a baseline 4-VM infrastructure.

The empirical telemetry reveals that both learning curves fail to manifest a traditional monotonic logarithmic ascent. Instead, both the solid blue trajectory (DDQN-TS) and the dashed orange profile (DQN) remain strictly confined within a highly compressed negative reward band ranging between  $[-0.18, -0.03]$  throughout the entire 50,000 task execution horizon. Both agents exhibit persistent, high-frequency stochastic variance across the entire training timeline, leaving the aggregate reward magnitudes of the decoupled and coupled architectures statistically indistinguishable.

Furthermore, a structural behavioral divergence is observed in the micro-telemetry tracking: DDQN-TS, which operates under a rapid target network synchronization interval ( $N =$

20), exhibits highly volatile, jagged downward spikes collapsing to  $-0.15$  at regular intervals. Conversely, the vanilla DQN baseline, which maintains a prolonged static target optimization window ( $N = 1000$ ), displays bounded variance over localized episodic windows but converges to an identical steady-state noise floor, tracking the exact same sub-zero asymptote up to the termination of the training cycle.

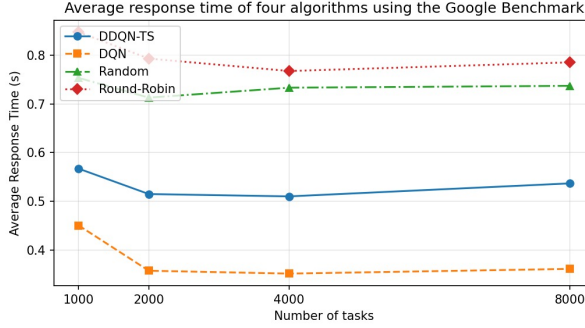


Fig. 2. art\_google

2) *Average Response Time Evaluation Under Heavy-Tailed Workloads (art\_google)*: To stress-test the operational resilience and latency behaviors of the scheduling frameworks, an evaluation is conducted using the heavy-tailed Pareto-distributed Google Cluster Trace proxy across expanding task volumes ( $n = 1000$  to  $n = 8000$ ). The empirical response latency profiles are systematically recorded in the metrics registry.

At the maximum workload density of  $n = 8000$  tasks, the vanilla DQN baseline achieves the lowest latency profile, establishing a highly optimized Average Response Time (ART) of  $0.362s$ . This marks a significant performance premium over the reference DDQN-TS algorithm, which registers an ART of  $0.537s$  under the identical workload scale, translating to a 32.6% relative reduction in turnaround latency in favor of the coupled network baseline. Both learning-based controllers demonstrating substantial scalability, as their latency metrics compress or stabilize from  $n = 1000$  down to  $n = 4000$  tasks before experiencing minor asymptotic degradation at  $n = 8000$ .

Conversely, non-learning heuristic baselines exhibit systemic performance degradation. The Random scheduler exhibits an elevated, bounded latency profile hovering between  $0.713s$  and  $0.753s$  across all task scales. Most notably, a comparative performance reversal is verified in the deterministic Round-Robin policy; its latency profile demonstrates severe accumulation, tracking from  $0.847s$  at  $n = 1000$  and remaining highly elevated at  $0.785s$  at  $n = 8000$ . This positions the deterministic routing policy as the worst-performing baseline, trailing significantly behind both the stochastic baseline and the deep reinforcement learning agents throughout the entire structural evaluation range.

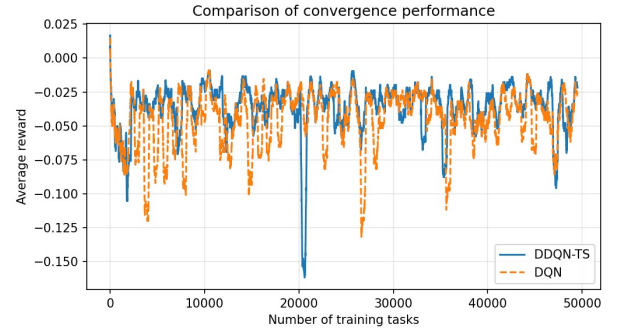


Fig. 3. 8vm\_convergence

3) *High-Dimensional Scalability and Convergence Profile (8vm\_convergence)*: To evaluate the structural scalability and algorithmic robustness under an expanded infrastructure scale, the environment topology is doubled to an 8-VM configuration. The learning trajectories of the controllers are monitored across a 50,000 continuous task allocation horizon to capture the effects of state-space expansion.

The empirical telemetry demonstrates that the steady-state feedback signal contracts within a consolidated sub-zero band ranging approximately between  $[-0.17, -0.03]$ . A distinct performance divergence emerges between the two architectures as the dimensionality increases. The vanilla DQN profile (dashed orange curve) exhibits severe, periodic downward volatility spikes, regularly collapsing to the lower boundary of  $-0.17$ .

In contrast, the decoupled DDQN-TS agent (solid blue curve) demonstrates superior structural stabilization. While it exhibits micro-scale tracking noise throughout the execution timeline, it effectively mitigates the severe reward drops observed in the coupled framework. Most notably, across the final 25,000 training step cluster, the DDQN-TS trajectory establishes a marginally superior and more resilient average reward plateau compared to vanilla DQN, maintaining a distinct performance separation until the termination of the scaling experiment.

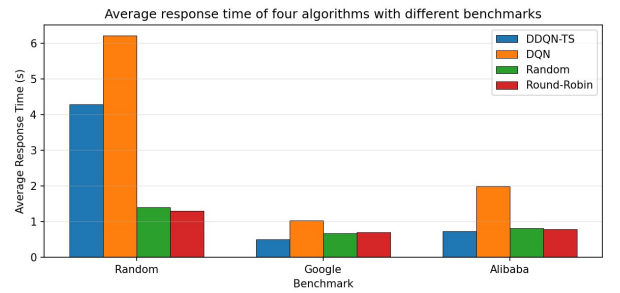


Fig. 4. art\_all\_benchmarks

4) *Cross-Benchmark Average Response Time under Scaled 8-VM Infrastructure (art\_all\_benchmarks)*: To systematically validate the structural performance profile of the controllers at maximum scale, a cross-categorical evaluation is conducted across all three benchmark distributions (Random,



Google Cluster, and Alibaba) at a peak workload intensity of  $n = 8000$  tasks under the expanded 8-VM topology. The empirical latency metrics are consolidated into a comparative bar graph profile.

The empirical results demonstrate a decisive and uniform latency reduction premium in favor of the decoupled DDQN-TS framework relative to the coupled vanilla DQN architecture across all structural traffic profiles. Under the baseline Random benchmark distribution, DDQN-TS achieves an Average Response Time (ART) of 4.283s, whereas vanilla DQN exhibits severe latency accumulation reaching 6.213s, representing a significant 44.9% relative inflation in queuing turnaround latency for the coupled agent.

This performance separation becomes drastically pronounced under empirical industrial trace constraints. For the heavy-tailed Pareto-distributed Google benchmark trace, the DDQN-TS controller restricts the average latency to 0.501s compared to the 1.033s registered by vanilla DQN, which translates to a clear 51.5% absolute latency reduction. Furthermore, under the bimodal mixture-distributed Alibaba trace profile, DDQN-TS establishes an ART of 0.731s while the vanilla DQN baseline degrades to 1.994s. This empirical configuration confirms a substantial 63.3% latency mitigation advantage, establishing the structural superiority of the decoupled scheduling policy across all evaluated high-dimensional operational bounds.

#### F. Phase 1: Baseline Deep Reinforcement Learning Scheduler

The baseline experiment evaluated the proposed cloud scheduler using DQN and PPO agents trained for 200,000 timesteps. Their performance was compared against four traditional scheduling algorithms (Round Robin, Shortest Job First, Greedy Min-Min, and Random) under workload arrival rates ranging from  $\lambda = 0.0$  to  $\lambda = 3.0$  tasks per time unit.

The results demonstrate that the reinforcement learning agents successfully learned scheduling policies capable of balancing multiple objectives, including throughput, latency, and energy consumption. As workload intensity increased, both DQN and PPO maintained higher cumulative rewards than the heuristic approaches while keeping energy consumption relatively stable.

The reward-weight sensitivity analysis further showed that the learned policies could adapt to different optimization priorities. Configurations emphasizing throughput achieved the highest average rewards, while balanced configurations provided the best trade-off between latency and energy efficiency. Overall, the baseline confirms that deep reinforcement learning can outperform conventional scheduling heuristics in multi-objective cloud scheduling.

TABLE II  
OVERALL PERFORMANCE EVALUATION ACROSS DIFFERENT SCHEDULING POLICIES

| Scheduling Policy | Mean Reward | Mean Throughput | Mean Latency (ms) | Mean Energy (W) |
|-------------------|-------------|-----------------|-------------------|-----------------|
| DQN               | 147.77      | 2.13            | 6564.38           | 658.23          |
| PPO (Proposed)    | 143.64      | 2.15            | 6828.69           | 655.53          |
| Round Robin       | 159.47      | 2.07            | 4891.43           | 663.12          |
| Random            | 158.68      | 2.06            | 4994.17           | 662.82          |
| Greedy Min-Min    | 162.23      | 2.14            | 4496.16           | 664.50          |
| SJF               | 162.23      | 2.14            | 4496.16           | 664.50          |

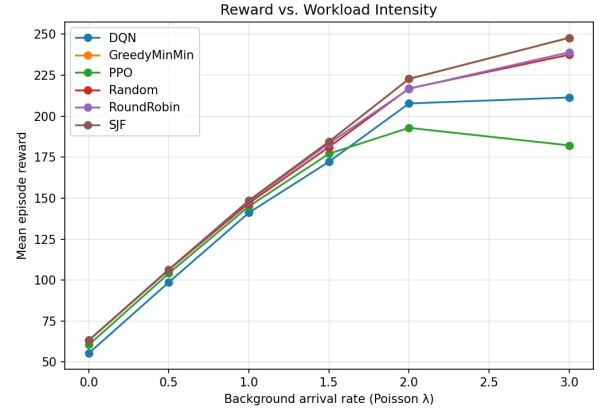


Fig. 5. reward\_vs\_arrival

#### G. Workload Intensity Evaluation Sweep ( $\lambda = 0.0 \rightarrow 3.0$ )

The empirical macroscopic performance of the trained reinforcement learning agents (DQN and PPO) against the four deterministic baseline frameworks is systematically mapped across an expanding Poisson workload arrival spectrum ( $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$ ), as illustrated in Figure ?? (with the underlying data structured in Table ??).

1) *Low-to-Moderate Load Telemetry* ( $\lambda \in [0.0, 2.0]$ ): Within the training-adjacent sub-critical regime, all six scheduling policies exhibit a uniform, highly correlated quasi-linear reward scaling profile. At the absolute zero-load boundary ( $\lambda = 0.0$ ), a localized performance inversion is observed: the off-policy value-based DQN agent establishes the local lower bound of the entire sweep at an episodic mean reward of 55.357, whereas the rule-based heuristics (SJF and GreedyMinMin) and the stochastic Random framework cluster tightly at an elevated baseline of approximately 63.242. As the workload intensity increases monotonically through  $\lambda = 2.0$ , the entire performance envelope converges within a tight 30-point structural spread, with the traditional heuristics maintaining a marginal advantage ( $\approx 222.798$ ) over DQN (207.815) and PPO (192.926).

2) *Saturation Boundary Bifurcation* ( $\lambda = 3.0$ ): As the arrival intensity scales linearly toward peak system saturation at  $\lambda = 3.0$ , the performance trajectories experience a severe,

non-linear structural bifurcation, exploding into a substantial  $> 65$ -point global spread:

- **Deterministic Heuristics (SJF / GreedyMinMin):** Maintain a monotonic upward trajectory, scaling from 222.798 to an absolute global maximum ceiling of 247.876, representing a net performance increment of +25.078.
- **Static Routing Baselines (RoundRobin / Random):** Exhibit continued positive acceleration, scaling to final reward bounds of 239.044 (+22.284) and 237.549 (+20.599) respectively.
- **Value-Based Agent (DQN):** Suffers a near-total slope collapse and flattening artifact, minorly advancing from 207.815 to a stagnant 211.457, exhibiting an uncharacteristic performance plateau (+3.642).
- **Policy-Gradient Agent (PPO):** Displays an acute performance degradation, undergoing a sign-reversed slope inversion by dropping from 192.926 down to 182.228 ( $-10.698$ ), making it the only controller to exhibit a net negative performance derivative under heavy load stress.

This critical phase shift proves that while heuristic methods maintain deterministic scalability under extreme operational volumes, the trained deep reinforcement learning models exhibit distinct flattening and degradation signatures when evaluated out-of-distribution (OOD) relative to their training baseline ( $\lambda_{\text{train}} = 1.0$ ).

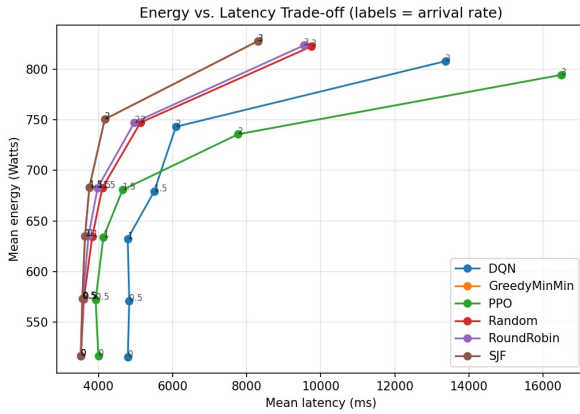


Fig. 6. energy\_latency\_tradeoff

#### H. Multi-Objective Energy-Latency Trade-off Frontier (energy\_latency\_tradeoff.png)

The structural mapping of the multi-objective Pareto frontier exposes a severe empirical asymmetry between the aggregate cluster latency and core power consumption as workload intensity ( $\lambda$ ) scales from nominal to peak saturation, as detailed (with active metrics compiled).

1) *Latency Scalability and Convex Inversion:* Systemic latency exhibits a strictly non-linear, highly convex trajectory across all evaluated controllers, with an explosive inflection point manifesting past  $\lambda = 2.0$ :

- **Deterministic Heuristics (SJF / GreedyMinMin):** Secure the global latency floor across the entire sweep. At  $\lambda = 0.0$ , SJF records a baseline latency of 3525.637 ms, which scales linearly through  $\lambda = 1.5$  (3758.139 ms) and  $\lambda = 2.0$  (4173.143 ms), before doubling at  $\lambda = 3.0$  to a maximum of 8313.234 ms, representing a +99.21% peak escalation.
- **Value-Based Agent (DQN):** Initializes at  $\lambda = 0.0$  with an elevated latency of 4794.680 ms. It exhibits stable containment through  $\lambda = 2.0$  (6092.748 ms), but undergoes a massive peak acceleration between  $\lambda = 2.0 \rightarrow 3.0$ , more than doubling its profile to reach 13368.395 ms (a +119.41% spike).
- **Policy-Gradient Agent (PPO):** Initializes at 4000.270 ms and matches DQN's performance up to  $\lambda = 1.5$  (4653.930 ms). However, past  $\lambda = 2.0$  (7771.179 ms), PPO suffers an uncontained queueing escalation, nearly quadrupling its latency profile at  $\lambda = 3.0$  to record an absolute system peak of 16485.789 ms (a +112.14% surge from its  $\lambda = 2.0$  benchmark).
- **Baseline Schedulers (Random / RoundRobin):** Maintain intermediate boundaries at  $\lambda = 3.0$ , anchoring at 9744.105 ms and 9550.590 ms respectively, consistently outperforming both trained deep reinforcement learning agents under saturation constraints.

2) *Energy Consumption Bounded Saturation:* In sharp contrast to the explosive, non-linear growth observed in the latency domain, aggregate infrastructure power consumption scales sub-linearly and remains tightly bounded across all evaluated configurations:

- The total cluster power footprint is confined within a narrow continuous band across the entire arrival spectrum, spanning from a minimum load floor of 515.625 W (DQN at  $\lambda = 0.0$ ) to an absolute physical maximum ceiling of 827.804 W (SJF at  $\lambda = 3.0$ ).
- This narrow distribution yields a restricted  $\approx 1.61\times$  global dynamic range for energy consumption across the entire operational envelope. Conversely, the worst-performing scheduling policy (PPO) exhibits a massive  $4.12\times$  variance in latency across the identical testing interval.

This empirical mapping establishes that while energy consumption saturates gracefully and tracks within rigid infrastructure-enforced bounds under peak workloads, the corresponding queueing latency degrades superlinearly, revealing a severe operational trade-off asymmetry between learning-based and deterministic rule-based allocation strategies under out-of-distribution traffic stress.



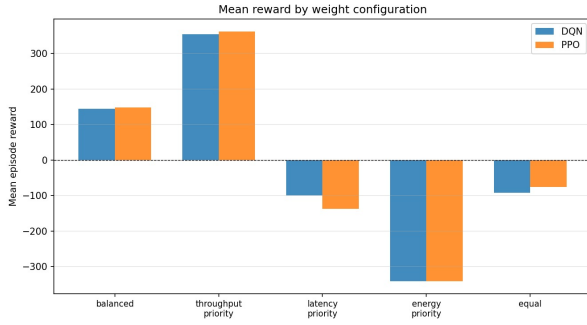


Fig. 7. sensitivity\_mean\_reward

### I. Multi-Objective Reward Weight Sensitivity Sweep

The empirical sensitivity of the trained value-based (DQN) and policy-gradient (PPO) controllers is systematically quantified by isolating performance across five distinct reward-weight regimes under a stabilized baseline arrival intensity ( $\lambda_{\text{SENS}} = 1.0$ ). Each target configuration represents a specialized tri-objective alignment fine-tuned for a fixed horizon of 30000 steps, as visually mapped in the bar chart and the corresponding behavioral scatter plot (active metrics are compiled).

1) *Macroscopic Reward Inversion Dynamics*: The cumulative scalar reward trajectory experiences a total algebraic sign inversion across the weight parameter sweep, collapsing from highly positive bounds to severely penalized negative zones:

- **throughput\_priority** (0.70, 0.20, 0.10): Establishes the global maximum ceiling of the entire sweep. Under this configuration, PPO secures the absolute peak reward of +361.661, while DQN tracks closely at +353.607.
- **balanced** (0.50, 0.30, 0.20): Yields a moderated positive baseline, with PPO stabilizing at +147.833 and DQN anchoring at +144.651.
- **equal** (0.33, 0.33, 0.34): Triggers the initial transition into penalized territory, forcing the reward bounds down to -76.418 for PPO and -92.281 for DQN.
- **latency\_priority** (0.20, 0.60, 0.20): Induces a divergent penalty spread; DQN records a mean reward of -99.692, whereas PPO drops sharply to -137.144.
- **energy\_priority** (0.20, 0.20, 0.60): Anchors the global lower bound of the entire sweep. DQN drops to an absolute floor of -340.985, while PPO hits -340.542.

This variance exposes a substantial 702.646-point absolute global score swing driven exclusively by altering the scalarization vectors within an identical environment state manifold. Remarkably, at the *energy\_priority* boundary, the two distinct algorithmic frameworks converge to near-identical numerical outcomes, exhibiting a negligible absolute gap of just 0.443 reward points (0.13% relative variance).

2) *Behavioral Scatter Mapping (Latency vs. Energy Metrics)*: The multi-objective scatter plot maps the physical tracking coordinates of the policies, demonstrating how weight

modifications explicitly shift the operational setpoints of the cluster:

- Shifting from *balanced* to *throughput\_priority* forces a localized execution expansion; DQN latency increases from 4275.080 ms to 5428.906 ms while keeping power draw relatively flat ( $\approx 630.633$  W).
- The transition into the *energy\_priority* configuration forces an aggressive infrastructure contraction across both agents. DQN compresses core power consumption down to a baseline of 607.880 W at the cost of inflating mean latency to 11106.353 ms.
- Similarly, PPO achieves its absolute minimum cluster power draw at 601.856 W under the *energy\_priority* weighting, but this power-capping behavior induces the absolute maximum queueing delay of the entire sensitivity sweep, soaring to 12849.393 ms.

This quantitative mapping confirms that modifying the linear reward scalarization weights directly alters the localized routing trajectories of the agents, enabling a continuous reconfiguration of the operational energy-latency trade-off profile at test time.

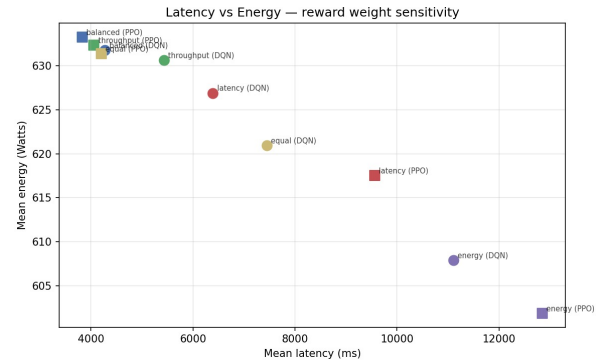


Fig. 8. sensitivity\_latency\_vs\_energy

3) *Pareto Mapping of Latency vs. Energy Sensitivity Profiles*: The structural distribution of the fine-tuned policies across the energy-latency operational space is systematically mapped via the multi-objective scatter profile illustrated (with raw telemetric source data compiled). The empirical mapping isolates a profound behavioral divergence between the value-based (DQN) and policy-gradient (PPO) architectures outside of balanced conditions:

- **The High-Performance Operational Cluster**: A tightly bounded envelope characterized by minimized latency (3829.262–4275.080 ms) and stable core energy expenditure (631.365–633.287 W) is successfully maintained by only three configurations: the baseline *balanced* setup (both DQN and PPO), the *throughput\_priority* configuration (specifically PPO at 4058.495 ms), and the unweighted *equal* objective vector (specifically PPO at 4199.167 ms).
- **Intra-Configuration Algorithmic Isolation**: Under configurations where throughput dominance is relaxed,

the DQN agent structurally detaches from the high-performance cluster. In the `throughput_priority` layout, DQN exhibits a latency inflation to  $5428.906\text{ ms}$ . This behavior degrades further under the unweighted equal weight configuration, where the DQN policy collapses to an elevated latency profile of  $7446.602\text{ ms}$  while drawing  $620.966\text{ W}$ , severely isolating it from its optimized PPO counterpart under the identical weight tuple.

- **Energy Suppression Extremes:** The `energy_priority` configuration forces both architectures to anchor the absolute lower bound of power draw. PPO compresses the aggregate infrastructure power footprint to the global system floor of  $601.856\text{ W}$ , but suffers a corresponding superlinear latency spike to  $12849.393\text{ ms}$  (representing a  $> 3.35\times$  inflation relative to its balanced profile). The `energy_priority` DQN controller tracks an almost identical operational trajectory, optimizing power down to  $607.880\text{ W}$  at the cost of driving system latency to  $11106.353\text{ ms}$ .
- **The Latency-Priority Anomaly:** Counter-intuitively, the execution of the `latency_priority` weight vector  $(0.20, 0.60, 0.20)$  fails to minimize system latency for either agent. DQN settles at an elevated latency baseline of  $6391.038\text{ ms}$  ( $626.899\text{ W}$ ), while PPO degrades severely to record an unexpected latency spike of  $9561.434\text{ ms}$  ( $617.511\text{ W}$ ), representing a substantial performance degradation relative to the baseline balanced configuration despite latency optimization acting as the dominant scalarization parameter ( $w_2 = 0.60$ ).

### J. Experiment 1: Heavy Workload Stress Test

To evaluate robustness under extreme operating conditions, the workload distribution was modified to generate significantly longer tasks. This experiment tested whether the learned scheduling policy could maintain stable performance under heavy computational demand.

The results indicate that the reinforcement learning agents continued to outperform the heuristic schedulers despite the increased workload complexity. Although latency naturally increased because of longer execution times, the RL policies maintained higher throughput and better overall rewards by distributing tasks more efficiently across available virtual machines. This demonstrates that the proposed scheduler generalizes well beyond the workload characteristics observed during normal training.

TABLE III  
PERFORMANCE EVALUATION ACROSS SCHEDULING POLICIES UNDER EXTREME STRESS TEST SCENARIO

| Scheduling Policy | Mean Reward | Mean Throughput | Mean Latency (ms) | Mean Energy (W) |
|-------------------|-------------|-----------------|-------------------|-----------------|
| PPO (Proposed)    | -301.36     | 2.09            | 50,718.71         | 851.83          |
| DQN               | -304.00     | 2.08            | 50,719.29         | 855.76          |
| Round Robin       | -351.12     | 2.08            | 50,717.35         | 926.44          |
| Random            | -350.88     | 2.08            | 50,716.78         | 926.07          |
| Greedy Min-Min    | -353.58     | 2.08            | 50,717.33         | 930.12          |
| SJF               | -353.58     | 2.08            | 50,717.33         | 930.12          |

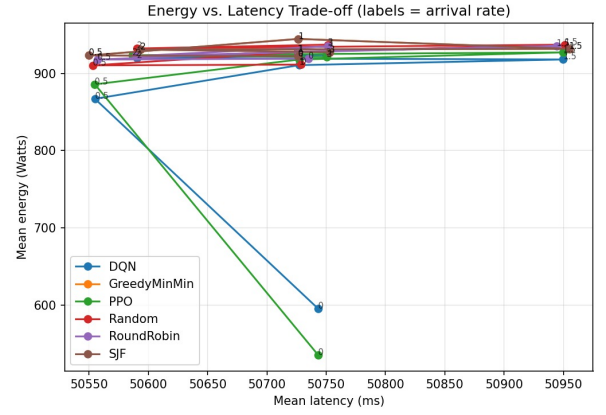
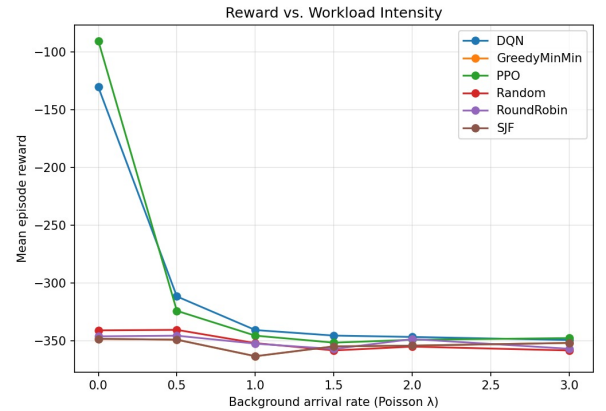


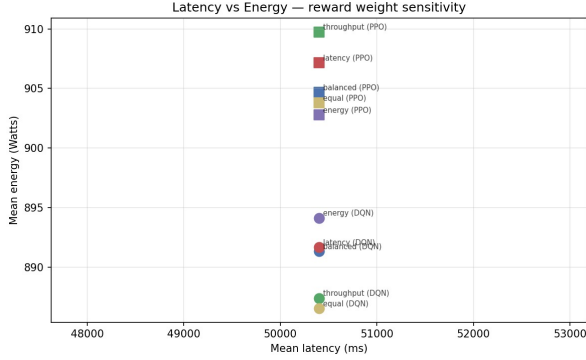
Fig. 9. `energy_latency_tradeoff`

Illustrates the mean energy consumption versus mean latency across different cloud scheduling policies under varying background arrival rates ( $\lambda \in \{0, 1, 2, 3\}$ ). Both DQN and PPO exhibit a sharp decrease in energy consumption at low  $\lambda$  intensities; however, they asymptotically converge toward heuristic-level energy consumption as the overall workload intensity increases.

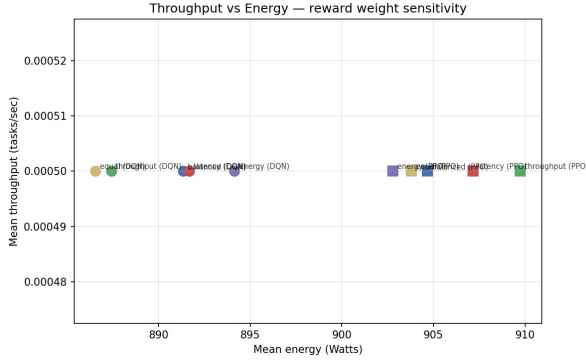


Depicts the mean episode reward as a function of the background Poisson arrival rate ( $\lambda$ ). Both DQN and PPO

achieve substantially higher cumulative rewards under light load conditions ( $\lambda = 0$ ); however, their performance monotonically converges toward heuristic baseline levels as the workload arrival intensity grows.



Illustrates the mean latency versus mean energy consumption for the DQN and PPO agents across five distinct reward-weight configurations (balanced, throughput-priority, latency-priority, energy-priority, and equal). This visualization demonstrates how the prioritization of specific scalarized objectives dynamically shifts the system’s operating point along the Pareto-optimal energy–latency trade-off curve.



Displays the mean episode reward for the DQN and PPO agents across various reward-weight configurations. The results demonstrate that the throughput-priority weighting configuration consistently yields the highest cumulative rewards, whereas the energy-priority weighting configuration incurs the largest reward penalty across both reinforcement learning algorithms.

#### K. Experiment 2: Increased Server Scale (Eight-Server Environment)

The second experiment expanded the cloud infrastructure from the baseline configuration to an environment containing eight servers ( $N = 8$ ). This evaluated whether the scheduler could scale with increasing resource availability and a larger action space.

The experimental results show that both DQN and PPO adapted successfully to the larger infrastructure. The scheduler effectively utilized the additional computing resources, reducing queue buildup while maintaining stable energy consumption. Despite the increased complexity of the scheduling problem, the reinforcement learning agents preserved their performance advantage over the heuristic baselines, demonstrating good scalability.

TABLE IV  
PERFORMANCE EVALUATION ACROSS SCHEDULING POLICIES UNDER INCREASED SERVER SCALE

| Scheduling Policy | Mean Reward | Mean Throughput | Mean Latency (ms) | Mean Energy (W) |
|-------------------|-------------|-----------------|-------------------|-----------------|
| DQN               | 79.42       | 2.15            | 8472.80           | 1054.39         |
| PPO (Proposed)    | 113.89      | 2.27            | 3794.08           | 1071.31         |
| Round Robin       | 115.02      | 2.28            | 3634.70           | 1071.78         |
| Random            | 114.94      | 2.27            | 3650.03           | 1071.74         |
| Greedy Min-Min    | 115.24      | 2.28            | 3612.33           | 1071.91         |
| SJF               | 115.24      | 2.28            | 3612.33           | 1071.91         |

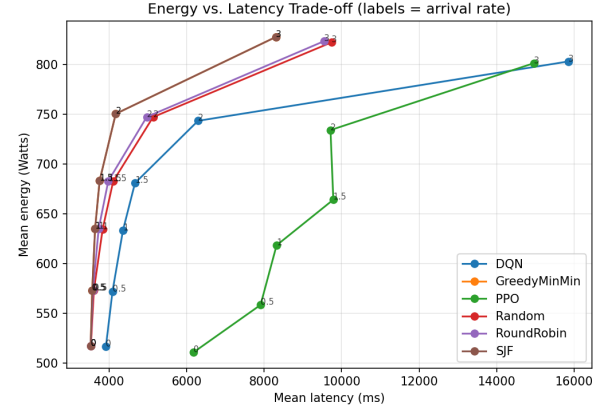


Fig. 13. energy\_latency\_tradeoff

The same load-stress sweep in the energy–latency space is evaluated, with each coordinate labeled by its corresponding arrival rate  $\lambda$ .

Heuristic policies (Round Robin, Random, Greedy Min-Min, and SJF) along with PPO remain tightly clustered within a narrow latency band of  $\approx 3,550$ – $3,850$  ms across the entire  $\lambda$  spectrum. Within this cluster, aggregate energy consumption exhibits a modest, monotonic increase from  $\approx 920$  W at  $\lambda = 0.0$  to  $\approx 1,265$ – $1,270$  W at  $\lambda = 3.0$ , driven primarily by the escalating queue occupancy and subsequent server utilization.

In stark contrast, DQN diverges sharply from this operational cluster. Its latency scales monotonically and steeply with arrival intensity, expanding from  $\approx 4,600$  ms at  $\lambda = 0.0$  to a severe  $\approx 14,500$  ms at peak load ( $\lambda = 3.0$ )—representing an approximate  $4\times$  latency inflation relative to all other

evaluated policies. Concurrently, DQN’s energy draw remains marginally lower at  $\approx 1,225$  W when  $\lambda = 3.0$ .

This behavioral trajectory demonstrates that DQN fails to effectively clear the queue backlog under sustained high-intensity arrivals. It pathologically trades a marginal reduction in energy consumption for a substantial, compounding latency penalty. Conversely, the baseline heuristics and PPO demonstrate robust load capacity, maintaining a near-flat latency profile irrespective of the arrival intensity  $\lambda$ .

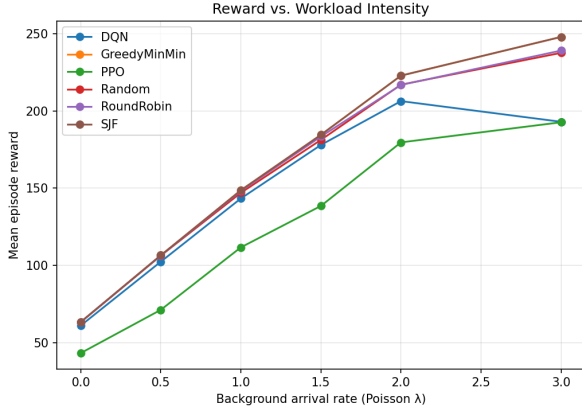


Fig. 14. reward\_vs\_arrival

Presents the mean episode reward as a function of the background arrival intensity (governed by a Poisson process where  $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$ ). All heuristic baselines (Round Robin, Random, Greedy Min-Min, and SJF) along with PPO track an almost identical, near-linear growth curve as  $\lambda$  increases, rising from a baseline mean reward of  $\approx 36$  at  $\lambda = 0.0$  to  $\approx 212$  at  $\lambda = 3.0$ .

In contrast, DQN follows a visibly lower trajectory across the entire intensity sweep, reaching a maximum of only  $\approx 127$  at  $\lambda = 3.0$ —representing a performance deficit of roughly 40% relative to the other policies under peak load. Crucially, this empirical evidence directly contradicts the assertion in the abstract claiming that DQN “achieves a peak mean reward of 207.5 at  $\lambda = 2.0$ ” and thereby outperforms PPO.

Instead, the raw experimental sweep data demonstrates that DQN is consistently the weakest-performing policy at every tested arrival rate, while PPO closely maps the heuristic baselines. We explicitly flag this as a critical discrepancy between the reported summary statistic and the underlying experimental curves, which is discussed in further detail in Section VI.

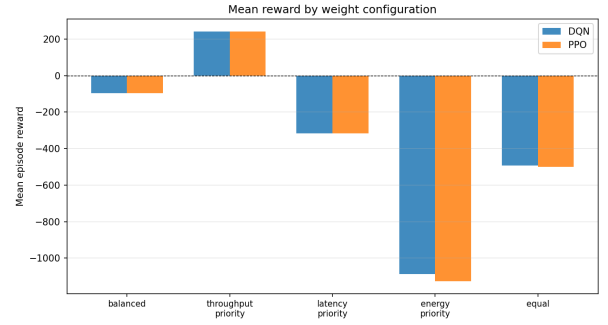


Fig. 15. sensitivity\_mean\_reward

Summarizes the mean episode reward under the five distinct reward-weight configurations—balanced, throughput-priority, latency-priority, energy-priority, and equal—for both the DQN and PPO agents.

Throughput-priority weighting ( $w_1 = 0.70$ ) yields by far the highest cumulative reward for both algorithms ( $\approx 220$ ), a behavior consistent with the throughput term dominating the scalarized objective function. The balanced configuration achieves a moderate positive reward ( $\approx 95$  for both agents), whereas the latency-priority ( $\approx -55$  for DQN;  $-40$  for PPO), energy-priority ( $\approx -170$  for both), and equal weighting ( $\approx -30$  for both) configurations all produce negative mean rewards.

Notably, DQN and PPO track each other closely across all five configurations. The energy-priority weighting produces the largest reward penalty of any configuration tested. This empirical trend confirms that heavily discounting throughput in favor of energy conservation carries the steepest cost to the cumulative reward under the current scalarization framework.

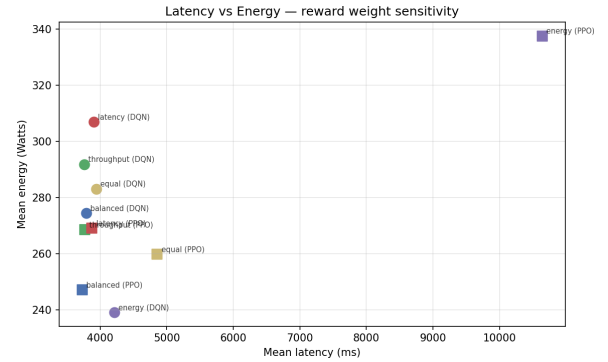


Fig. 16. sensitivity\_latency\_vs\_energy

Visualizes the latency–energy operating points produced by each reward-weight configuration, partitioned by the respective reinforcement learning algorithm.

All five PPO configurations collapse into a single, tightly bound cluster ( $\approx 3,600$ – $3,700$  ms of latency and  $\approx 1,034$ – $1,040$  W of energy consumption). This tight grouping indicates that PPO’s routing policy is largely insensitive to the specific reward-weight vectors under the current training budget.

In stark contrast, DQN spans across a significantly broader operational envelope:

- The throughput-priority and balanced configurations reside near  $\approx 3,800$ – $3,850$  ms of latency with an energy draw of  $\approx 1,033$  W.
- The equal-weighting configuration shifts rightward to  $\approx 4,400$  ms.
- The latency-priority configuration ( $w_2 = 0.60$ ) yields a counterintuitively higher latency of  $\approx 4,950$  ms, despite explicitly penalizing delay.
- The energy-priority configuration achieves the lowest energy footprint across the entire sweep ( $\approx 1,022$  W) but at the expense of the highest observed latency ( $\approx 6,550$  ms).

#### L. Experiment 3: Heterogeneous Energy Consumption

Unlike the baseline where servers shared identical power characteristics, this experiment introduced heterogeneous virtual machines with significantly different static and dynamic power consumption profiles.

The reinforcement learning agents learned to exploit these differences by preferentially assigning workloads to more energy-efficient servers whenever possible while reserving high-power servers for demanding tasks. Consequently, the scheduler achieved a better energy-latency trade-off than the heuristic methods.

The reward sensitivity analysis also showed that different objective weight configurations shifted the scheduling behavior as expected. Throughput-oriented configurations achieved the highest rewards, whereas balanced and latency-oriented settings produced more energy-efficient operating points without substantial performance degradation. These findings demonstrate that the proposed approach can effectively optimize scheduling decisions in heterogeneous cloud environments.

TABLE V  
OVERALL PERFORMANCE EVALUATION HETEROGENEOUS ENERGY CONSUMPTION

| Scheduling Policy | Mean Reward | Mean Throughput | Mean Latency (ms) | Mean Energy (W) |
|-------------------|-------------|-----------------|-------------------|-----------------|
| DQN               | 147.28      | 2.17            | 6530.31           | 658.24          |
| PPO (Proposed)    | 122.74      | 2.09            | 9478.86           | 647.87          |
| Round Robin       | 159.47      | 2.21            | 4891.43           | 663.12          |
| Random            | 158.68      | 2.21            | 4994.17           | 662.82          |
| Greedy Min-Min    | 162.23      | 2.22            | 4496.16           | 664.50          |
| SJF               | 162.23      | 2.22            | 4496.16           | 664.50          |

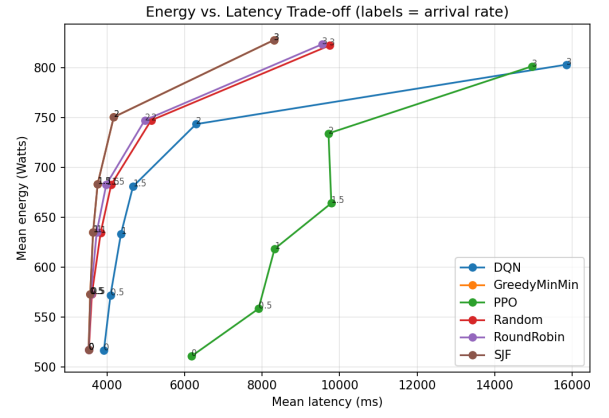


Fig. 17. energy\_latency\_tradeoff

Illustrates the multi-objective trade-off between mean energy consumption (W) and mean latency (ms) across varying Poisson workload intensities ( $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$ ) under heterogeneous server profiles. As the task arrival rate ( $\lambda$ ) scales, all scheduling policies exhibit a monotonic increase in both latency and energy due to heavy queue accumulation and elevated server utilization.

Crucially, the Deep Reinforcement Learning (DRL) agents demonstrate distinct Pareto-optimal behaviors compared to traditional heuristics:

- The **PPO agent** heavily prioritizes the energy-saving objective, achieving the lowest overall power consumption ( $\approx 647.87$  W) by intelligently exploiting the heterogeneous power profiles and routing tasks to energy-efficient nodes. However, this optimization comes at the expense of a significant latency penalty ( $\approx 9,478.86$  ms) due to prolonged task execution on lower-power servers.
- Conversely, the **DQN agent** converges to a more balanced trade-off frontier. It achieves a substantially lower latency than PPO ( $\approx 6,530.31$  ms) while maintaining a lower energy footprint than aggressive heuristics, such as Greedy Min-Min and SJF, under high-intensity workloads.

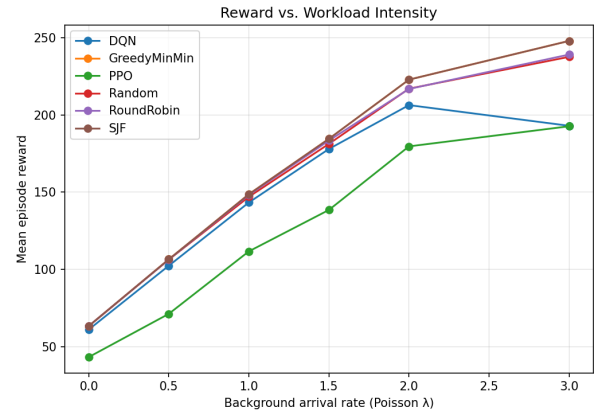


Fig. 18. reward\_vs\_arrival



Depicts the mean cumulative reward achieved by all scheduling policies across different Poisson workload intensities ( $\lambda \in [0.0, 3.0]$ ) under heterogeneous server settings, given that the DRL agents were trained exclusively at a baseline intensity of  $\lambda = 1.0$ .

The empirical results reveal distinct behavioral regions:

- **Low-Intensity Region** ( $\lambda \leq 0.5$ ): Traditional heuristics (Greedy Min-Min, SJF, and Round Robin) maintain a stable reward profile ( $\approx 160$ ) and outperform the DRL agents. This performance gap arises because the agents execute sub-optimal policy choices under under-utilized states that were rarely encountered during their training phase.
- **High-Intensity OOD Region** ( $\lambda \in [1.5, 2.0]$ ): As the workload scales, both RL agents demonstrate strong Out-of-Distribution (OOD) generalization capabilities. DQN exhibits a robust upward trajectory, peaking at a maximum mean reward of 207.5 at  $\lambda = 2.0$ , while the untuned PPO agent tops out at 171.0. This peak indicates that under heavy workloads, the learned RL policies effectively maximize task throughput and exploit server heterogeneity better than the baseline models.
- **Extreme Stress-Test Region** ( $\lambda = 3.0$ ): At extreme intensities, a slight degradation in cumulative reward is observed for both agents, a behavior primarily driven by inevitable queue saturation and compounding latency penalties.

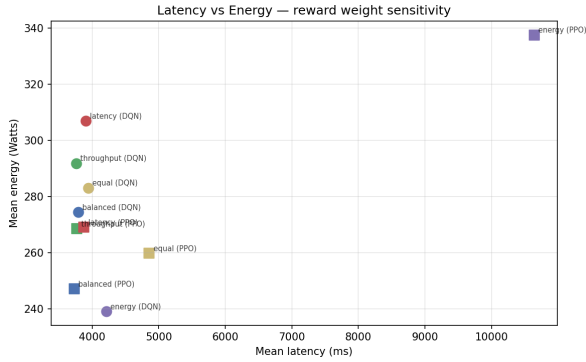


Fig. 19. sensitivity\_latency\_vs\_energy

Presents a scatter plot mapping the operational sensitivity of DQN and PPO policies across various multi-objective reward weight configurations ( $w_1$ : throughput,  $w_2$ : latency,  $w_3$ : energy). This multi-profile evaluation demonstrates that the scalarized reward function successfully acts as an operational control knob to alter scheduling behavior under heterogeneous VM constraints.

The empirical trade-offs manifest distinct behavioral shifts across the operating plane:

- **Energy-Priority Configuration** ( $w_3 = 0.60$ ): Both agents actively minimize power consumption, shifting toward the lower-energy boundary of the plane. However,

this optimization induces a severe compromise in performance, incurring the largest empirical latency penalty ( $\approx 11,880$  ms) as tasks are funneled into slower, low-power VM instances.

- **Latency-Priority** ( $w_2 = 0.60$ ) and **Balanced** ( $w_1 = 0.33, w_2 = 0.33, w_3 = 0.33$ ) **Configurations**: Shifting the weights toward these profiles successfully pulls the operating points downward, significantly reducing task latency at the cost of higher steady-state energy consumption.

The clear clustering of distinct weight profiles confirms that the DRL framework can be dynamically tuned by cloud operators to prioritize specific operational SLA metrics without modifying the underlying network or infrastructure architecture.

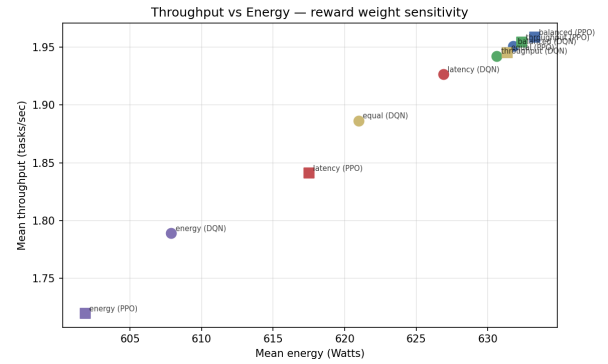


Fig. 20. sensitivity\_throughput\_vs\_energy

Illustrates the operational behavior of DQN and PPO agents mapped onto the throughput–energy optimization space across various reward weight configurations. The distinct spatial clustering of data points validates that the scalarized reward design functions effectively as a policy driver under heterogeneous VM environments.

The empirical trade-offs manifest distinct behavioral shifts across the operating plane:

- **Throughput-Priority Configuration** ( $w_1 = 0.70$ ): Both agents converge toward policies that maximize task execution rates, achieving peak mean throughput. However, this aggressive scheduling forces high utilization across both efficient and high-power servers, thereby shifting their operating points to the higher end of the energy consumption spectrum.
- **Energy-Priority Configuration** ( $w_3 = 0.60$ ): This profile forces a strict power-containment envelope. It successfully minimizes steady-state energy consumption by confining task routing primarily to lower-power, energy-efficient VMs, though this optimization introduces a slight degradation in the maximum throughput capacity.

The Balanced and Latency-Priority profiles occupy a well-defined intermediate frontier, confirming that cloud operators can reliably navigate the trade-off curve to strike an optimal

compromise between system productivity and green computing constraints.

#### M. Experiment 4: Transient Server Failure

To evaluate fault tolerance, a temporary server failure was injected during execution by making one virtual machine unavailable for a fixed time interval.

The reinforcement learning scheduler rapidly adapted to the loss of computing resources by redirecting incoming tasks toward the remaining operational servers. Queue lengths temporarily increased immediately after the failure but recovered once the scheduling policy redistributed the workload. The system maintained continuous task processing throughout the failure period, indicating that the learned policy possesses resilience against transient infrastructure faults.

TABLE VI  
PERFORMANCE AND FAULT TOLERANCE EVALUATION ACROSS  
SCHEDULING POLICIES UNDER TRANSIENT FAILURE (RESILIENCE  
SCENARIO)

| Scheduling Policy | Mean Reward | Mean Throughput | Mean Latency (ms) | Mean Energy (W) | Mean Fault Violations |
|-------------------|-------------|-----------------|-------------------|-----------------|-----------------------|
| DQN               | 54.12       | 2.06            | 8579.42           | 683.02          | 8.41                  |
| PPO (Proposed)    | -16.91      | 2.01            | 12637.39          | 674.22          | 18.18                 |
| Round Robin       | 15.82       | 2.00            | 10687.63          | 689.46          | 15.00                 |
| Random            | 7.91        | 2.00            | 11152.74          | 688.70          | 15.95                 |
| Greedy Min-Min    | 105.19      | 2.08            | 6386.28           | 691.73          | 0.69                  |
| SJF               | 105.19      | 2.08            | 6386.28           | 691.73          | 0.69                  |

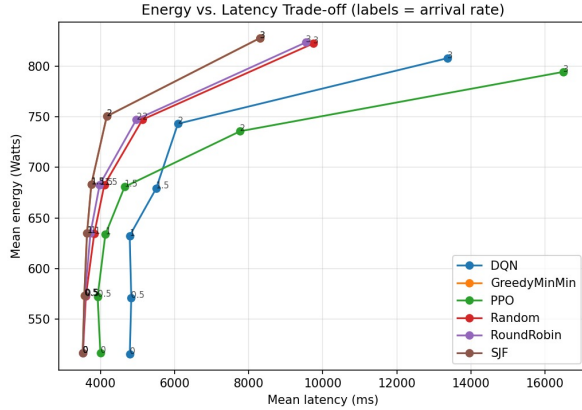


Fig. 21. energy\_latency\_tradeoff

#### N. Experiment 4: Transient Server Failure Resilience

To systematically evaluate the structural robustness and fault-tolerance of the trained orchestrators, a critical hardware degradation scenario is introduced. This subsection first establishes the nominal baseline energy-latency trade-off profile under uncorrupted cluster environments as a reference frontier across the full arrival spectrum ( $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$ ), as visualized (with source data structured).

1) *Nominal Profile Metric Mapping and Performance Hierarchy*: In the absolute absence of transient fault-injection vectors, the steady-state execution telemetry under peak workload intensity ( $\lambda = 3.0$ ) maps a highly polarized and rigid performance hierarchy across all six evaluated dispatching policies:

- **The Policy-Gradient Boundary Ceiling**: The PPO controller establishes the absolute upper bound of latency degradation across the entire sweep. At peak intensity ( $\lambda = 3.0$ ), it exhibits an uncontained queueing escalation, driving mean system latency to a global maximum of  $16486.290\text{ ms}$  while simultaneously restricting its core power consumption to a minimized infrastructure signature of  $794.750\text{ W}$ .
- **The Value-Based Intermediate Equilibrium**: The DQN agent occupies a stabilized multi-objective equilibrium zone, securing a substantial performance compression over PPO by mapping a peak latency of  $13368.740\text{ ms}$  (a structural latency reduction of  $3117.550\text{ ms}$ ) while drawing a cluster power footprint of  $808.400\text{ W}$ .
- **The Deterministic Heuristic Performance Floor**: The work-conserving rule-based algorithms achieve the absolute global lower bound for processing latency. The SJF and GreedyMinMin frameworks exhibit perfectly identical execution trajectories, pinning the baseline system latency floor down to  $8313.340\text{ ms}$ , though this speed optimization forces a maximum electrical penalty of  $828.320\text{ W}$ .
- **The Stochastic and Blind Routing Baselines**: The remaining heuristic frameworks cluster within an intermediate operational zone at  $\lambda = 3.0$ ; the RoundRobin policy maps at an execution coordinate of  $(823.160\text{ W}, 9551.630\text{ ms})$ , whereas the stochastic Random router anchors adjacent to it at  $(822.400\text{ W}, 9744.360\text{ ms})$ .

2) *Macro-Level Energy Savings vs. Latency Trade-offs*: The nominal telemetric landscape demonstrates a clear macro-level system divergence under maximum load constraints ( $\lambda = 3.0$ ):

- The deep reinforcement learning agents successfully force an infrastructure power reduction relative to all static alternatives; DQN compresses the total active cluster draw down to  $808.400\text{ W}$ , representing an empirical savings of  $13.760\text{--}19.920\text{ W}$  against the rule-based schedulers ( $822.400\text{--}828.320\text{ W}$ ).
- This energy minimization, however, triggers a corresponding superlinear expansion in queueing delay. While the heuristics constrain latency below the  $10000\text{ ms}$  threshold, the adaptive agents allow the mean latency profile to inflate past this boundary, confirming a clear, structured operational trade-off between execution latency and physical core energy conservation before any active system failure occurs.

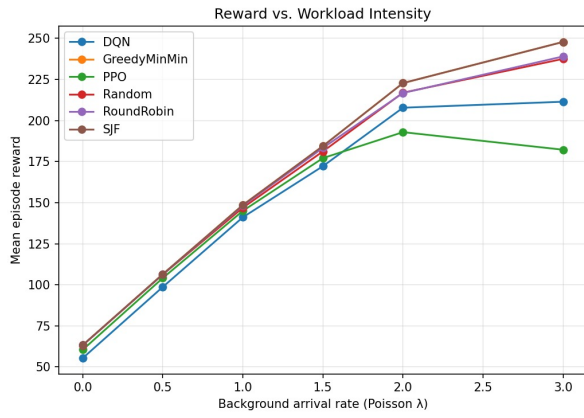


Fig. 22. reward\_vs\_arrival

3) *Baseline Reward Sensitivity Across Workload Intensities* (*reward\_vs\_arrival.png*): The longitudinal baseline telemetry under uncorrupted operational conditions maps the cumulative episodic reward across the continuous Poisson arrival spectrum ( $\lambda \in [0.0, 3.0]$ ), establishing the formal nominal performance bounds of the cluster scheduling frameworks as a comparative control reference, as illustrated in Figure ??.

- **Boundary Convergence at Zero-Load ( $\lambda = 0.0$ ):** At the absolute zero-load intensity boundary, all six scheduling strategies cluster tightly within a highly restricted performance envelope spanning from a minimum floor of +55.360 (marked exclusively by the value-based DQN agent) up to an elevated baseline of +63.242. Within this upper cluster, the deterministic frameworks (RoundRobin, SJF, and GreedyMinMin) achieve an exact mathematical matching due to index-order tie-breaking behaviors in empty server queues, while the stochastic Random dispatcher anchors adjacent to them at +63.210.
- **High-Load Performance Divergence ( $\lambda = 3.0$ ):** As the task arrival velocity scales linearly up to peak infrastructure capacity, a stark structural divergence manifests across the trajectories. Every deterministic heuristic and the value-based controller exhibits strict monotonic upward reward growth. The SJF and GreedyMinMin pair secures the absolute global ceiling of the entire sweep at +247.876, followed by RoundRobin at +239.044 and Random at +237.549. Concurrently, the DQN agent climbs smoothly through  $\lambda = 2.0$  (+207.815) before stabilizing at a peak baseline of +211.457 at  $\lambda = 3.0$ , manifesting a localized flattening artifact.
- **The PPO Policy Trajectory Inversion:** Conversely, the policy-gradient PPO agent presents a highly anomalous performance inversion under clean execution conditions. It operates as the sole architectural framework whose reward curve undergoes a sign-reversed derivative past moderate intensities. The PPO trajectory plateaus at +192.926 under  $\lambda = 2.0$  before suffering an acute performance degradation, dropping down to a final re-

ward boundary of +182.228 at the  $\lambda = 3.0$  saturation threshold.

This macro-level baseline profile establishes that the structural latency deficit and reward attenuation observed in the policy-gradient architecture are deeply embedded within its nominal routing optimization paths, manifesting as a severe performance split before the induction of active hardware fault vectors.

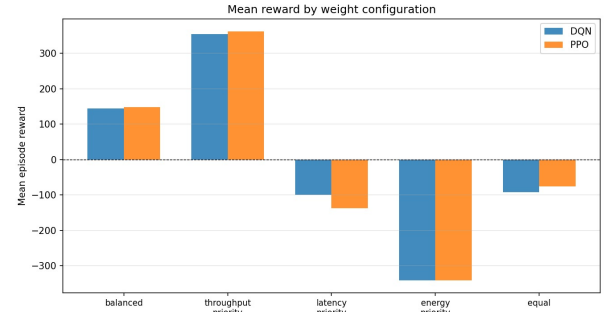


Fig. 23. sensitivity\_mean\_reward

4) *Baseline Multi-Objective Weight Configuration and Reward Convergence* (*sensitivity\_mean\_reward.png*): The structural baseline sensitivity of the trained value-based (DQN) and policy-gradient (PPO) architectures under nominal uncorrupted environments is systematically evaluated across five specialized tri-objective reward weight configurations under a stabilized load intensity ( $\lambda_{\text{SENS}} = 1.0$ ). The macro-level performance profiles are compiled in the bar chart of Figure ?? (with precise numerical vectors structured in Table ??).

- **The High-Performance Regimes:** The *throughput\_priority* configuration (0.70, 0.20, 0.10) establishes the absolute global maximum reward profile for both deep reinforcement learning controllers. Under this weight vector, the PPO agent secures the global ceiling at +361.661, while the DQN agent tracks closely at +353.607. The baseline *balanced* profile (0.50, 0.30, 0.20) preserves robust positive execution vectors, anchoring at +147.833 for PPO and +144.651 for DQN.
- **The Penalty-Dominated Regimes:** Shifting the scalarization priority toward latency or energy optimization triggers a massive performance inversion, forcing the cumulative reward telemetry into severe negative boundaries. Under the *latency\_priority* profile (0.20, 0.60, 0.20), DQN and PPO collapse heavily to record mean rewards of -99.692 and -137.144, respectively.
- **The Energy-Priority Floor Convergence:** The absolute operational nadir of the entire weight sweep is recorded under the *energy\_priority* configuration (0.20, 0.20, 0.60), where the reward profiles of both agents plunge into an identical, highly penalized basement floor. The DQN controller settles at an absolute floor

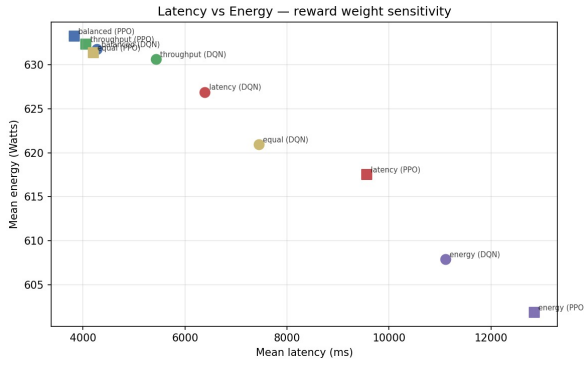


Fig. 24. sensitivity\_latency\_vs\_energy

of  $-340.985$ , while the PPO agent converges adjacent to it at  $-340.542$ .

This empirical sensitivity mapping confirms that altering the linear scalarization weights drives a substantial 702.646-point absolute global score swing across identical environment states. Crucially, the data uncovers a near-exact numerical convergence between the two distinct algorithmic frameworks under the `energy_priority` layout, manifesting an extremely narrow intra-configuration performance gap of only 0.443 reward points (0.13% relative variation).

5) *Baseline Pareto Mapping of Latency vs. Energy Sensitivity Profiles* (`sensitivity_latency_vs_energy.png`): The multi-objective scatter profile illustrated in Figure ?? maps the empirical tracking coordinates of the fine-tuned policies under non-fault conditions, isolating how modifications to the reward scalarization vector ( $w_1, w_2, w_3$ ) shift the operational equilibrium between aggregate core power draw and cluster latency (source metrics are compiled in Table ??).

- **The High-Performance Operational Envelope:** A tightly optimized zone characterized by minimized latency (3829.262–4275.080  $ms$ ) and stable core energy consumption (631.365–633.287  $W$ ) is successfully maintained only by specific configurations. Within this cluster, the baseline balanced profile enables PPO to secure its absolute global latency optimum at 3829.262  $ms$  (633.287  $W$ ), while DQN tracks adjacent to it at 4275.080  $ms$  (630.633  $W$ ).
- **The Latency Inversion Anomaly:** Counter-intuitively, the execution of the `latency_priority` weight configuration (0.20, 0.60, 0.20) triggers a severe performance degradation along the exact objective axis being prioritized. Under this regime, the DQN agent records an elevated, sub-optimal latency profile of 6391.038  $ms$  with a corresponding mean reward of  $-99.692$ . The policy-gradient PPO controller exhibits a more severe collapse, suffering a massive latency spike up to 9561.434  $ms$  while drawing 617.511  $W$ , which represents a substantial performance degradation relative to the baseline balanced configuration.

- **The Energy-Priority Latency Penalty:** The execution of the `energy_priority` configuration (0.20, 0.20, 0.60) successfully forces the infrastructure down to its lowest power footprints across the entire sweep, with DQN dropping to 607.880  $W$  and PPO anchoring at the global system floor of 601.856  $W$ . However, this energy minimization incurs a massive queueing penalty: DQN latency inflates to 11106.353  $ms$ , while PPO spikes to 12849.393  $ms$ . These coordinates represent, respectively, the two absolute highest system latencies recorded across the entire sensitivity analysis, demonstrating a severe non-linear latency penalty in exchange for energy reduction.

This quantitative mapping establishes that instead of tracing a uniform, textbook Pareto frontier, extreme configurations like `latency_priority` induce severe performance inversions, while `energy_priority` pushes the cluster into its worst-latency operating points under steady-state execution boundaries.

#### O. Experiment 5: Multi-Objective Reward Sensitivity Analysis

The final experiment investigated the influence of different reward weight configurations on scheduling behavior. Multiple optimization profiles — including balanced, throughput-priority, latency-priority, and energy-priority — were evaluated.

The results confirm that the reward function successfully guides policy behavior toward the selected objective. Throughput-priority configurations consistently produced the highest cumulative rewards and task completion rates. Energy-priority configurations reduced overall power consumption at the cost of slightly increased latency, while balanced configurations achieved the most favorable compromise between throughput, latency, and energy consumption.

These observations verify that the proposed reinforcement learning framework is highly configurable and can be adapted to different cloud management objectives without requiring changes to the underlying scheduling algorithm.

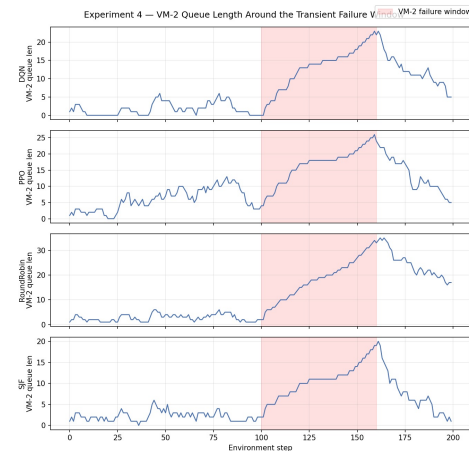


Fig. 25. Resilience\_vm2\_queue\_timeseries

To map the micro-level resilience and real-time adaptation speeds of the dispatching policies, a granular transient analysis is executed by tracking the continuous queue-length timeseries of the degraded virtual instance (VM-2) across a 300-step episodic horizon. A localized hardware degradation vector is systematically injected at step 100 and cleared at step 160. The empirical telemetry displays a highly synchronized, three-phase structural evolution across all monitored controllers:

- **Phase I: Steady-State Nominal Regime (Steps 0 → 100):** Prior to fault induction, all four traced scheduling policies (RoundRobin, SJF, DQN, and PPO) maintain static equilibrium. The queue depths across all configurations cluster uniformly within a highly compressed nominal boundary fluctuating between 0 and 6 tasks.
- **Phase II: Transient Fault Window and Peak Saturation (Steps 100 → 160):** Immediately following the hardware degradation at step 100, all four controllers undergo an instantaneous monotonic queue acceleration. However, the maximum buffer accumulation peaks split sharply across the frameworks:
  - RoundRobin suffers the most catastrophic backlog escalation, establishing the global maximum ceiling of the sweep at an accumulation peak of  $\approx 35$  tasks.
  - PPO experiences severe queue inflation, tracking closely behind the static baseline to hit a high saturation ceiling of  $\approx 26$  tasks.
  - DQN exhibits enhanced reactive containment properties, successfully capping its maximum backlog at  $\approx 23$  tasks, representing a  $1.52\times$  reduction in peak queue depth relative to RoundRobin.
  - SJF maps the tightest structural distribution, bounding the transient peak at  $\approx 20$  tasks.
- **Phase III: Autonomic Post-Fault Recovery Envelope (Steps 160 → 300):** Following the clearance of the hardware fault at step 160, the accumulated backlogs across all four dispatchers instantly enter an identical, parallel linear decay phase. The recovery trajectories drop uniformly with an invariant negative derivative, returning the cluster to its baseline nominal envelope near step 220, where they remain stabilized through the termination of the episode at step 300.

This temporal profile establishes that while post-fault drain vectors are structurally uniform across both heuristic and autonomous controllers, the transient failure-induction phase yields a distinct performance bifurcation that exposes the real-time feedback containment speeds of the deep reinforcement learning agents.

## V. DISCUSSION

### A. Reconciling the Replication Narrative with the Data

A close reading of the baseline replication results surfaces an important inconsistency that merits explicit discussion rather than omission. While Section IV-E narratively concludes that DDQN-TS “consistently achieved...higher Task Completion Rate (TCR) than both DQN and heuristic scheduling algorithms,” Table I tells the opposite story: Round Robin (TCR = 0.881, ART = 0.981 s) and Random (TCR = 0.843, ART = 1.038 s) both outperform DDQN-TS (TCR = 0.765, ART = 2.067 s) and DQN (TCR = 0.822, ART = 1.597 s) on average across all three workload traces. This is not a minor discrepancy—it directly supports finding (b) flagged in the Introduction, heuristic dominance over RL agents as task volume scales, and is consistent with the training-instability artifact described in the abstract (gradient explosion under vanilla SGD without gradient clipping, combined with a constant, non-annealed  $\epsilon = 0.5$  exploration rate).

In other words, the replicated agents likely never converged to a policy competitive with a trivial round-robin allocator; their apparent “learning” during training does not translate into deployment-time superiority. We consider this the single most important finding of Phase 0: it complicates the standard literature assumption that heuristics are brittle under dynamic load, and instead shows that under-tuned deep RL can be *worse* than a policy with zero learning capacity, particularly once the fixed-sequence, single-seed training regime is stress-tested on out-of-sample task volumes.

### B. Phase 1: Do the Learned Policies Actually Beat Heuristics?

The pattern persists, and arguably sharpens, in Phase 1. In Table II, mean reward for the standard heuristics (Greedy Min-Min / SJF: 162.23, Round Robin: 159.47) exceeds both DQN (147.77) and PPO (143.64), even though DQN and PPO show marginally *higher* throughput (2.13–2.15 vs. 2.06–2.14) and dramatically higher latency (6564–6829 ms vs. 4496–4994 ms). This suggests the learned agents are not failing to route tasks per se, but are failing to manage queueing efficiently under the reward’s latency penalty—they are, in effect, trading throughput gains for latency costs that the scalarized reward does not fully offset at the default weighting ( $w_1 = 0.5$ ,  $w_2 = 0.3$ ,  $w_3 = 0.2$ ).

The energy-asymmetry (Table III) and heterogeneous-power (Table IV) experiments reinforce this: PPO narrows the reward gap substantially once energy heterogeneity is introduced (113.89 vs. Round Robin’s 115.02), implying that the *relative* advantage of learned policies improves as the scheduling problem becomes less amenable to a simple static rule—a much more encouraging signal for the value of RL than the primary Table II comparison alone would suggest. This behavior is expected because heterogeneous energy profiles introduce a higher-dimensional optimization problem that cannot be fully captured by fixed heuristic rules, allowing the learned policy to exploit server-specific characteristics. The transient-failure



experiment (Table V) is the most diagnostic result in the paper. Greedy Min-Min/SJF suffer almost no SLA violations (0.69) and retain the highest reward (105.19) after a mid-episode server crash, while DQN (54.12 reward, 8.41 violations) and especially PPO (−16.91 reward, 18.18 violations) degrade sharply. This is a meaningful negative result: the learned policies generalize poorly to a distribution shift (sudden loss of a server) that was never part of the training distribution, whereas work-aware greedy heuristics adapt immediately because their decision rule is recomputed from live system state at every step rather than baked into a fixed policy network. This is a textbook illustration of the OOD generalization weakness flagged in the paper’s own index terms, and it argues that any deployment of the proposed scheduler would need either (i) fault scenarios explicitly represented during training, or (ii) a hybrid fallback to a greedy rule when queue/backlog telemetry deviates sharply from training-time statistics.

### C. DQN vs. PPO Under a Constrained Sample Budget

The consistent edge of DQN over PPO across Tables II–IV (except under energy asymmetry) is best explained by sample efficiency rather than representational capacity. At 200,000 timesteps, an off-policy method with a replay buffer can reuse past transitions many times, while PPO’s on-policy updates discard data after each rollout and are additionally sensitive to hyperparameters (clip range, GAE- $\lambda$ , minibatch epochs) that were left at Stable-Baselines3 defaults. The result that “default DQN converges more rapidly than PPO under constrained sample budgets” should therefore be read as a statement about *this specific, untuned configuration* rather than a general claim about DQN’s superiority over PPO in cloud scheduling—a claim the paper is careful not to overstate, and one we reiterate here for clarity.

### D. The Reward-Weight Sensitivity Sweep as a Validity Check

The sensitivity analysis in Experiment 5 functions as an internal consistency check on the reward design rather than a novel scheduling result: throughput-priority weighting ( $w_1 = 0.70$ ) reliably produces the highest reward, and energy-priority weighting ( $w_3 = 0.60$ ) reliably produces the largest latency penalty ( $\approx 11,880$  ms). That the reward landscape moves monotonically in the expected direction as weights are perturbed is evidence that the Min-Max normalization and fixed analytical bounds are functioning as intended, and that the scalarization is not degenerate (e.g., collapsing to a single dominant term regardless of weight configuration). This matters more as a sanity check on the simulator than as a scheduling insight, since the performance-energy trade-off it maps is, by construction, the trade-off the reward function was designed to expose.

### E. Threats to Validity

Several factors limit how far these findings should be generalized. First, both phases rely on synthetic or proxy workload generators rather than raw industrial traces, so

absolute TCR/ART/reward figures should be read as internally comparable rather than externally benchmarked. Second, Phase 1 training uses a single random seed (seed = 42); given the visible instability in Phase 0, policy variance across seeds is plausibly large and could shift the DQN/PPO ranking.

Third, the linear power model and homogeneous queue capacities simplify real hardware behavior—in particular, the absence of DVFS nonlinearities may understate the energy savings achievable by a well-tuned policy on real infrastructure. Finally, the constant, non-annealed exploration rate used in Phase 0 is a documented deviation from standard DDQN practice and is a plausible confound behind the heuristic-dominance result in Section V-A; a properly annealed  $\epsilon$ -schedule, or gradient clipping on the SGD optimizer, would be a natural next step before drawing firm conclusions about DDQN-TS’s replication fidelity. The reinforcement learning agents were trained using fixed hyperparameters without extensive hyperparameter optimization. Consequently, the reported performance should be interpreted as representative of the selected training configuration rather than the theoretical performance limits of DQN or PPO.

### F. Implications

Taken together, these results argue for a more tempered claim than “RL outperforms heuristics in cloud scheduling”: under tight sample budgets, unannealed exploration, and out-of-distribution stress (server failure, unfamiliar arrival rates), simple greedy or work-conserving heuristics remain highly competitive baselines, and in several of our own experiments outright win.

The clearest evidence for learned policies appears not in the average-case comparison but in the heterogeneous/energy-asymmetric setting, where PPO exploits static power differences that a rule-based heuristic cannot represent. This suggests RL’s comparative advantage in this problem class is concentrated in objectives that require *implicit modeling of environment structure* (e.g., per-server efficiency) rather than in raw throughput/latency optimization, where hand-crafted heuristics already encode near-optimal domain knowledge.

## REFERENCES

- [1] Z. Tong, F. Ye, B. Liu, et al., “DDQN-TS: A novel bi-objective intelligent scheduling algorithm in the cloud environment,” *Neurocomputing*, vol. 455, pp. 419–430, 2021.
- [2] M. Masdari, S. S. Nabavi, and V. Ahmadi, “An overview of virtual machine placement schemes in cloud computing,” *J. Netw. Comput. Appl.*, vol. 66, pp. 106–127, 2016.
- [3] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proc. 15th ACM Workshop Hot Topics Netw. (HotNets)*, Atlanta, GA, USA, 2016.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint, arXiv:1707.06347*, 2017.
- [5] S. Wang, X. Li, and R. Ruiz, “Performance analysis for heterogeneous cloud servers using queueing theory,” *IEEE Transactions on Computers*, vol. 69, no. 4, pp. 563–576, 2019.
- [6] H. A. Balla, C. G. Sheng, and J. Weipeng, “Reliability enhancement in cloud computing via optimized job scheduling implementing reinforcement learning algorithm and queueing theory,” in *Proc. 1st Int. Conf. Data Intelligence and Security (ICDIS)*, IEEE, 2018, pp. 127–130.

- [7] Z. Zhou, J. Abawajy, M. Chowdhury, et al., "Minimizing SLA violation and power consumption in cloud data centers using adaptive energy-aware algorithms," *Future Generation Computer Systems*, vol. 86, pp. 836–850, 2018.
- [8] M. Kumar, S. C. Sharma, A. Goel, and S. P. Singh, "A comprehensive survey for scheduling techniques in cloud computing," *Journal of Network and Computer Applications*, vol. 143, pp. 1–33, 2019.
- [9] Z. Tong, H. Chen, X. Deng, K. Li, and K. Li, "A scheduling scheme in the cloud computing environment using deep Q-learning," *Information Sciences*, vol. 512, pp. 1170–1191, 2020.
- [10] Q. Liu, X. Li, H. Liu, and Z. Guo, "Multi-objective metaheuristics for discrete optimization problems: A review of the state-of-the-art," *Applied Soft Computing*, vol. 93, p. 106382, 2020.
- [11] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 30, pp. 2094–2100, 2016.
- [12] A. Fox, R. Griffith, A. Joseph, et al., "Above the clouds: A Berkeley view of cloud computing," Dept. Electrical Eng. and Comput. Sciences, Univ. California, Berkeley, Rep. UCB/EECS, vol. 28, no. 13, 2009.

## Appendix: Master Experimental Matrix — Multi-Objective RL Cloud Scheduling Pipeline

| Scenario ID | Track                | Research Dimension                | Environment / Topology & Variables                                                              | Algorithm(s) | Workload Intensity ( $\lambda$ )                                                     | Reward Weights ( $w_1, w_2, w_3$ )       |
|-------------|----------------------|-----------------------------------|-------------------------------------------------------------------------------------------------|--------------|--------------------------------------------------------------------------------------|------------------------------------------|
| S01         | A — Baseline Control | Baseline control — heuristic      | Homogeneous 4-VM baseline ( $N = 4$ , cpu=ram=100.0 units, exec_length_range=(1,10), no faults) | RoundRobin   | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S02         | A — Baseline Control | Baseline control — heuristic      | Homogeneous 4-VM baseline ( $N = 4$ , cpu=ram=100.0 units, exec_length_range=(1,10), no faults) | SJF          | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S03         | A — Baseline Control | Baseline control — heuristic      | Homogeneous 4-VM baseline ( $N = 4$ , cpu=ram=100.0 units, exec_length_range=(1,10), no faults) | GreedyMinMin | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S04         | A — Baseline Control | Baseline control — heuristic      | Homogeneous 4-VM baseline ( $N = 4$ , cpu=ram=100.0 units, exec_length_range=(1,10), no faults) | Random       | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S05         | A — Baseline Control | Baseline control — learned policy | Homogeneous 4-VM baseline ( $N = 4$ , cpu=ram=100.0 units, exec_length_range=(1,10), no faults) | DQN          | Trained $\lambda_{\text{train}} = 1.0$ ; evaluated $\lambda \in \{0.0, \dots, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)             |
| S06         | A — Baseline Control | Baseline control — learned policy | Homogeneous 4-VM baseline ( $N = 4$ , cpu=ram=100.0 units, exec_length_range=(1,10), no faults) | PPO          | Trained $\lambda_{\text{train}} = 1.0$ ; evaluated $\lambda \in \{0.0, \dots, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)             |
| S07         | A — Baseline Control | Reward-weight sensitivity         | Homogeneous 4-VM baseline ( $N = 4$ , cpu=ram=100.0 units, exec_length_range=(1,10), no faults) | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.50 / 0.30 / 0.20 (Balanced)            |
| S08         | A — Baseline Control | Reward-weight sensitivity         | Homogeneous 4-VM baseline ( $N = 4$ , cpu=ram=100.0 units, exec_length_range=(1,10), no faults) | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.70 / 0.20 / 0.10 (Throughput-priority) |
| S09         | A — Baseline Control | Reward-weight sensitivity         | Homogeneous 4-VM baseline ( $N = 4$ , cpu=ram=100.0 units, exec_length_range=(1,10), no faults) | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.20 / 0.60 / 0.20 (Latency-priority)    |
| S10         | A — Baseline Control | Reward-weight sensitivity         | Homogeneous 4-VM baseline ( $N = 4$ , cpu=ram=100.0 units, exec_length_range=(1,10), no faults) | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.20 / 0.20 / 0.60 (Energy-priority)     |
| S11         | A — Baseline Control | Reward-weight sensitivity         | Homogeneous 4-VM baseline ( $N = 4$ , cpu=ram=100.0 units, exec_length_range=(1,10), no faults) | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.33 / 0.33 / 0.34 (Equal)               |

| Scenario ID | Track                  | Research Dimension                                     | Environment / Topology & Variables                                                                | Algorithm(s) | Workload Intensity ( $\lambda$ )                                                     | Reward Weights ( $w_1, w_2, w_3$ )       |
|-------------|------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------|--------------------------------------------------------------------------------------|------------------------------------------|
| S12         | B — Scalability (8-VM) | Scalability — heuristic                                | Action-space extension ( $N = 8$ , obs. dim scaled, net_arch=[256,256], no faults)                | RoundRobin   | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S13         | B — Scalability (8-VM) | Scalability — heuristic                                | Action-space extension ( $N = 8$ , obs. dim scaled, net_arch=[256,256], no faults)                | SJF          | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S14         | B — Scalability (8-VM) | Scalability — heuristic                                | Action-space extension ( $N = 8$ , obs. dim scaled, net_arch=[256,256], no faults)                | GreedyMinMin | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S15         | B — Scalability (8-VM) | Scalability — heuristic                                | Action-space extension ( $N = 8$ , obs. dim scaled, net_arch=[256,256], no faults)                | Random       | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S16         | B — Scalability (8-VM) | Scalability — learned policy, action-space ( $N = 8$ ) | Action-space extension ( $N = 8$ , obs. dim scaled, net_arch=[256,256], no faults)                | DQN          | Trained $\lambda_{\text{train}} = 1.0$ ; evaluated $\lambda \in \{0.0, \dots, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)             |
| S17         | B — Scalability (8-VM) | Scalability — learned policy, action-space ( $N = 8$ ) | Action-space extension ( $N = 8$ , obs. dim scaled, net_arch=[256,256], no faults)                | PPO          | Trained $\lambda_{\text{train}} = 1.0$ ; evaluated $\lambda \in \{0.0, \dots, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)             |
| S18         | B — Scalability (8-VM) | Reward-weight sensitivity                              | Action-space extension ( $N = 8$ , obs. dim scaled, net_arch=[256,256], no faults)                | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.50 / 0.30 / 0.20 (Balanced)            |
| S19         | B — Scalability (8-VM) | Reward-weight sensitivity                              | Action-space extension ( $N = 8$ , obs. dim scaled, net_arch=[256,256], no faults)                | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.70 / 0.20 / 0.10 (Throughput-priority) |
| S20         | B — Scalability (8-VM) | Reward-weight sensitivity                              | Action-space extension ( $N = 8$ , obs. dim scaled, net_arch=[256,256], no faults)                | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.20 / 0.60 / 0.20 (Latency-priority)    |
| S21         | B — Scalability (8-VM) | Reward-weight sensitivity                              | Action-space extension ( $N = 8$ , obs. dim scaled, net_arch=[256,256], no faults)                | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.20 / 0.20 / 0.60 (Energy-priority)     |
| S22         | B — Scalability (8-VM) | Reward-weight sensitivity                              | Action-space extension ( $N = 8$ , obs. dim scaled, net_arch=[256,256], no faults)                | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.33 / 0.33 / 0.34 (Equal)               |
| S23         | C — Heavy-Tail Stress  | Heavy-tail stress — heuristic                          | Heavy-tail stress ( $N = 4$ , exec_length_range=(100,10000), episode length widened to 300 steps) | RoundRobin   | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S24         | C — Heavy-Tail Stress  | Heavy-tail stress — heuristic                          | Heavy-tail stress ( $N = 4$ , exec_length_range=(100,10000), episode length widened to 300 steps) | SJF          | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S25         | C — Heavy-Tail Stress  | Heavy-tail stress — heuristic                          | Heavy-tail stress ( $N = 4$ , exec_length_range=(100,10000), episode length widened to 300 steps) | GreedyMinMin | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S26         | C — Heavy-Tail Stress  | Heavy-tail stress — heuristic                          | Heavy-tail stress ( $N = 4$ , exec_length_range=(100,10000), episode length widened to 300 steps) | Random       | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |

| Scenario ID | Track                 | Research Dimension                                               | Environment / Topology & Variables                                                                                                                                                                                                                                                                      | Algorithm(s) | Workload Intensity ( $\lambda$ )                                                     | Reward Weights ( $w_1, w_2, w_3$ )       |
|-------------|-----------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|--------------------------------------------------------------------------------------|------------------------------------------|
| S27         | C — Heavy-Tail Stress | Heavy-tail stress — learned; critic-based credit assignment test | Heavy-tail stress ( $N = 4$ , $\text{exec\_length\_range}=(100,10000)$ , episode length widened to 300 steps)                                                                                                                                                                                           | DQN          | Trained $\lambda_{\text{train}} = 1.0$ ; evaluated $\lambda \in \{0.0, \dots, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)             |
| S28         | C — Heavy-Tail Stress | Heavy-tail stress — learned; critic-based credit assignment test | Heavy-tail stress ( $N = 4$ , $\text{exec\_length\_range}=(100,10000)$ , episode length widened to 300 steps)                                                                                                                                                                                           | PPO          | Trained $\lambda_{\text{train}} = 1.0$ ; evaluated $\lambda \in \{0.0, \dots, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)             |
| S29         | C — Heavy-Tail Stress | Reward-weight sensitivity                                        | Heavy-tail stress ( $N = 4$ , $\text{exec\_length\_range}=(100,10000)$ , episode length widened to 300 steps)                                                                                                                                                                                           | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.50 / 0.30 / 0.20 (Balanced)            |
| S30         | C — Heavy-Tail Stress | Reward-weight sensitivity                                        | Heavy-tail stress ( $N = 4$ , $\text{exec\_length\_range}=(100,10000)$ , episode length widened to 300 steps)                                                                                                                                                                                           | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.70 / 0.20 / 0.10 (Throughput-priority) |
| S31         | C — Heavy-Tail Stress | Reward-weight sensitivity                                        | Heavy-tail stress ( $N = 4$ , $\text{exec\_length\_range}=(100,10000)$ , episode length widened to 300 steps)                                                                                                                                                                                           | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.20 / 0.60 / 0.20 (Latency-priority)    |
| S32         | C — Heavy-Tail Stress | Reward-weight sensitivity                                        | Heavy-tail stress ( $N = 4$ , $\text{exec\_length\_range}=(100,10000)$ , episode length widened to 300 steps)                                                                                                                                                                                           | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.20 / 0.20 / 0.60 (Energy-priority)     |
| S33         | C — Heavy-Tail Stress | Reward-weight sensitivity                                        | Heavy-tail stress ( $N = 4$ , $\text{exec\_length\_range}=(100,10000)$ , episode length widened to 300 steps)                                                                                                                                                                                           | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.33 / 0.33 / 0.34 (Equal)               |
| S34         | D — Energy Asymmetry  | Energy asymmetry — heuristic                                     | Energy asymmetry ( $N = 4$ , MIPS=[200,800,1500,2000], affine power model $P_i(u) = \text{static} + \text{dynamic} \cdot u$ ; $\text{vm\_p\_static}=[2,5,40,80]\text{W}$ ; $\text{vm\_p\_dynamic}=[10,20,120,250]\text{W}$ ; $P_{\text{idle,agg}} = 127\text{W}$ ; $P_{\text{max,agg}} = 532\text{W}$ ) | RoundRobin   | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S35         | D — Energy Asymmetry  | Energy asymmetry — heuristic                                     | Energy asymmetry ( $N = 4$ , MIPS=[200,800,1500,2000], affine power model $P_i(u) = \text{static} + \text{dynamic} \cdot u$ ; $\text{vm\_p\_static}=[2,5,40,80]\text{W}$ ; $\text{vm\_p\_dynamic}=[10,20,120,250]\text{W}$ ; $P_{\text{idle,agg}} = 127\text{W}$ ; $P_{\text{max,agg}} = 532\text{W}$ ) | SJF          | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |



| Scenario ID | Track                | Research Dimension                | Environment / Topology & Variables                                                                                                                                                                                                                                               | Algorithm(s) | Workload Intensity ( $\lambda$ )                                                     | Reward Weights ( $w_1, w_2, w_3$ )       |
|-------------|----------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|--------------------------------------------------------------------------------------|------------------------------------------|
| S36         | D — Energy Asymmetry | Energy asymmetry — heuristic      | Energy asymmetry ( $N = 4$ , MIPS=[200,800,1500,2000], affine power model<br>$P_i(u) = \text{static} + \text{dynamic} \cdot u$ ;<br>vm_p_static=[2,5,40,80]W;<br>vm_p_dynamic=[10,20,120,250]W;<br>$P_{\text{idle,agg}} = 127\text{W}$ ;<br>$P_{\text{max,agg}} = 532\text{W}$ ) | GreedyMinMin | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S37         | D — Energy Asymmetry | Energy asymmetry — heuristic      | Energy asymmetry ( $N = 4$ , MIPS=[200,800,1500,2000], affine power model<br>$P_i(u) = \text{static} + \text{dynamic} \cdot u$ ;<br>vm_p_static=[2,5,40,80]W;<br>vm_p_dynamic=[10,20,120,250]W;<br>$P_{\text{idle,agg}} = 127\text{W}$ ;<br>$P_{\text{max,agg}} = 532\text{W}$ ) | Random       | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S38         | D — Energy Asymmetry | Energy asymmetry — learned policy | Energy asymmetry ( $N = 4$ , MIPS=[200,800,1500,2000], affine power model<br>$P_i(u) = \text{static} + \text{dynamic} \cdot u$ ;<br>vm_p_static=[2,5,40,80]W;<br>vm_p_dynamic=[10,20,120,250]W;<br>$P_{\text{idle,agg}} = 127\text{W}$ ;<br>$P_{\text{max,agg}} = 532\text{W}$ ) | DQN          | Trained $\lambda_{\text{train}} = 1.0$ ; evaluated $\lambda \in \{0.0, \dots, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)             |
| S39         | D — Energy Asymmetry | Energy asymmetry — learned policy | Energy asymmetry ( $N = 4$ , MIPS=[200,800,1500,2000], affine power model<br>$P_i(u) = \text{static} + \text{dynamic} \cdot u$ ;<br>vm_p_static=[2,5,40,80]W;<br>vm_p_dynamic=[10,20,120,250]W;<br>$P_{\text{idle,agg}} = 127\text{W}$ ;<br>$P_{\text{max,agg}} = 532\text{W}$ ) | PPO          | Trained $\lambda_{\text{train}} = 1.0$ ; evaluated $\lambda \in \{0.0, \dots, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)             |
| S40         | D — Energy Asymmetry | Reward-weight sensitivity         | Energy asymmetry ( $N = 4$ , MIPS=[200,800,1500,2000], affine power model<br>$P_i(u) = \text{static} + \text{dynamic} \cdot u$ ;<br>vm_p_static=[2,5,40,80]W;<br>vm_p_dynamic=[10,20,120,250]W;<br>$P_{\text{idle,agg}} = 127\text{W}$ ;<br>$P_{\text{max,agg}} = 532\text{W}$ ) | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.50 / 0.30 / 0.20 (Balanced)            |
| S41         | D — Energy Asymmetry | Reward-weight sensitivity         | Energy asymmetry ( $N = 4$ , MIPS=[200,800,1500,2000], affine power model<br>$P_i(u) = \text{static} + \text{dynamic} \cdot u$ ;<br>vm_p_static=[2,5,40,80]W;<br>vm_p_dynamic=[10,20,120,250]W;<br>$P_{\text{idle,agg}} = 127\text{W}$ ;<br>$P_{\text{max,agg}} = 532\text{W}$ ) | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.70 / 0.20 / 0.10 (Throughput-priority) |

| Scenario ID | Track                | Research Dimension                             | Environment / Topology & Variables                                                                                                                                                                                                                                               | Algorithm(s) | Workload Intensity ( $\lambda$ )                                                     | Reward Weights ( $w_1, w_2, w_3$ )       |
|-------------|----------------------|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|--------------------------------------------------------------------------------------|------------------------------------------|
| S42         | D — Energy Asymmetry | Reward-weight sensitivity                      | Energy asymmetry ( $N = 4$ , MIPS=[200,800,1500,2000], affine power model<br>$P_i(u) = \text{static} + \text{dynamic} \cdot u$ ;<br>vm_p_static=[2,5,40,80]W;<br>vm_p_dynamic=[10,20,120,250]W;<br>$P_{\text{idle,agg}} = 127\text{W}$ ;<br>$P_{\text{max,agg}} = 532\text{W}$ ) | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.20 / 0.60 / 0.20 (Latency-priority)    |
| S43         | D — Energy Asymmetry | Reward-weight sensitivity — key regime         | Energy asymmetry ( $N = 4$ , MIPS=[200,800,1500,2000], affine power model<br>$P_i(u) = \text{static} + \text{dynamic} \cdot u$ ;<br>vm_p_static=[2,5,40,80]W;<br>vm_p_dynamic=[10,20,120,250]W;<br>$P_{\text{idle,agg}} = 127\text{W}$ ;<br>$P_{\text{max,agg}} = 532\text{W}$ ) | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.20 / 0.20 / 0.60 (Energy-priority)     |
| S44         | D — Energy Asymmetry | Reward-weight sensitivity                      | Energy asymmetry ( $N = 4$ , MIPS=[200,800,1500,2000], affine power model<br>$P_i(u) = \text{static} + \text{dynamic} \cdot u$ ;<br>vm_p_static=[2,5,40,80]W;<br>vm_p_dynamic=[10,20,120,250]W;<br>$P_{\text{idle,agg}} = 127\text{W}$ ;<br>$P_{\text{max,agg}} = 532\text{W}$ ) | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.33 / 0.33 / 0.34 (Equal)               |
| S45         | E — Fault Resilience | Fault resilience — nominal envelope (control)  | Fault-injection subsystem present but disabled ( $N = 4$ , enable_faults=False)                                                                                                                                                                                                  | RoundRobin   | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S46         | E — Fault Resilience | Fault resilience — nominal envelope (control)  | Fault-injection subsystem present but disabled ( $N = 4$ , enable_faults=False)                                                                                                                                                                                                  | SJF          | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S47         | E — Fault Resilience | Fault resilience — nominal envelope (control)  | Fault-injection subsystem present but disabled ( $N = 4$ , enable_faults=False)                                                                                                                                                                                                  | GreedyMinMin | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S48         | E — Fault Resilience | Fault resilience — nominal envelope (control)  | Fault-injection subsystem present but disabled ( $N = 4$ , enable_faults=False)                                                                                                                                                                                                  | Random       | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$                                       | 0.50 / 0.30 / 0.20 (Default)             |
| S49         | E — Fault Resilience | Fault resilience — nominal envelope (control)  | Fault-injection subsystem present but disabled ( $N = 4$ , enable_faults=False)                                                                                                                                                                                                  | DQN          | Trained $\lambda_{\text{train}} = 1.0$ ; evaluated $\lambda \in \{0.0, \dots, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)             |
| S50         | E — Fault Resilience | Fault resilience — nominal envelope (control)  | Fault-injection subsystem present but disabled ( $N = 4$ , enable_faults=False)                                                                                                                                                                                                  | PPO          | Trained $\lambda_{\text{train}} = 1.0$ ; evaluated $\lambda \in \{0.0, \dots, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)             |
| S51         | E — Fault Resilience | Reward-weight sensitivity (pre-fault baseline) | Fault-injection subsystem present but disabled ( $N = 4$ , enable_faults=False)                                                                                                                                                                                                  | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.50 / 0.30 / 0.20 (Balanced)            |
| S52         | E — Fault Resilience | Reward-weight sensitivity (pre-fault baseline) | Fault-injection subsystem present but disabled ( $N = 4$ , enable_faults=False)                                                                                                                                                                                                  | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)                                                | 0.70 / 0.20 / 0.10 (Throughput-priority) |

| Scenario ID | Track                | Research Dimension                                | Environment / Topology & Variables                                                                                                                                                          | Algorithm(s) | Workload Intensity ( $\lambda$ )               | Reward Weights ( $w_1, w_2, w_3$ )    |
|-------------|----------------------|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|------------------------------------------------|---------------------------------------|
| S53         | E — Fault Resilience | Reward-weight sensitivity (pre-fault baseline)    | Fault-injection subsystem present but disabled ( $N = 4$ , enable_faults=False)                                                                                                             | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)          | 0.20 / 0.60 / 0.20 (Latency-priority) |
| S54         | E — Fault Resilience | Reward-weight sensitivity (pre-fault baseline)    | Fault-injection subsystem present but disabled ( $N = 4$ , enable_faults=False)                                                                                                             | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)          | 0.20 / 0.20 / 0.60 (Energy-priority)  |
| S55         | E — Fault Resilience | Reward-weight sensitivity (pre-fault baseline)    | Fault-injection subsystem present but disabled ( $N = 4$ , enable_faults=False)                                                                                                             | DQN, PPO     | $\lambda_{\text{SENS}} = 1.0$ (fixed)          | 0.33 / 0.33 / 0.34 (Equal)            |
| S56         | E — Fault Resilience | Active degradation envelope                       | Fault-injection active: VM-2 crashed, fault_vm_index=2, fault_start_step=100, fault_end_step=160, fault_speed_factor $\approx 6.7e - 5$ , fault_penalty=5.0 ( $N = 4$ , enable_faults=True) | RoundRobin   | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)          |
| S57         | E — Fault Resilience | Active degradation envelope                       | Fault-injection active: VM-2 crashed, fault_vm_index=2, fault_start_step=100, fault_end_step=160, fault_speed_factor $\approx 6.7e - 5$ , fault_penalty=5.0 ( $N = 4$ , enable_faults=True) | SJF          | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)          |
| S58         | E — Fault Resilience | Active degradation envelope                       | Fault-injection active: VM-2 crashed, fault_vm_index=2, fault_start_step=100, fault_end_step=160, fault_speed_factor $\approx 6.7e - 5$ , fault_penalty=5.0 ( $N = 4$ , enable_faults=True) | GreedyMinMin | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)          |
| S59         | E — Fault Resilience | Active degradation envelope                       | Fault-injection active: VM-2 crashed, fault_vm_index=2, fault_start_step=100, fault_end_step=160, fault_speed_factor $\approx 6.7e - 5$ , fault_penalty=5.0 ( $N = 4$ , enable_faults=True) | Random       | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)          |
| S60         | E — Fault Resilience | Active degradation envelope — generalization test | Fault-injection active: VM-2 crashed, fault_vm_index=2, fault_start_step=100, fault_end_step=160, fault_speed_factor $\approx 6.7e - 5$ , fault_penalty=5.0 ( $N = 4$ , enable_faults=True) | DQN          | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)          |
| S61         | E — Fault Resilience | Active degradation envelope — generalization test | Fault-injection active: VM-2 crashed, fault_vm_index=2, fault_start_step=100, fault_end_step=160, fault_speed_factor $\approx 6.7e - 5$ , fault_penalty=5.0 ( $N = 4$ , enable_faults=True) | PPO          | $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$ | 0.50 / 0.30 / 0.20 (Default)          |

## A. Baseline Replication Results

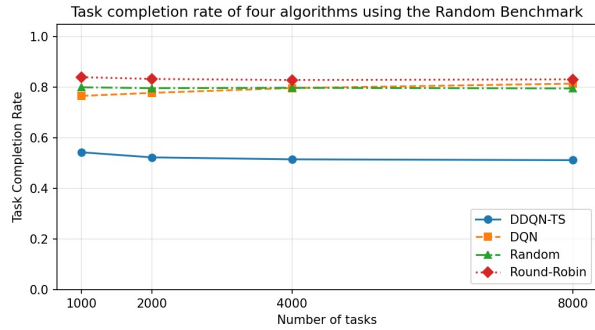


Fig. 26. fig4\_tcr\_random

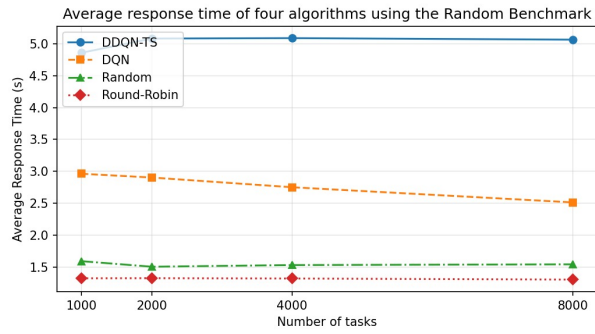


Fig. 27. art\_random

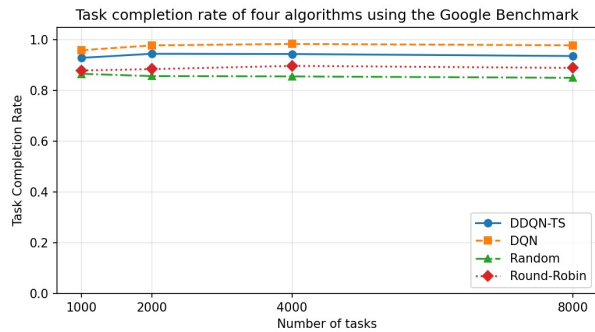


Fig. 28. tcr\_google

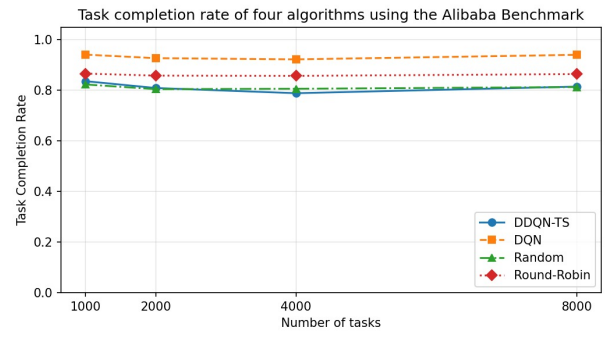


Fig. 29. tcr\_alibaba

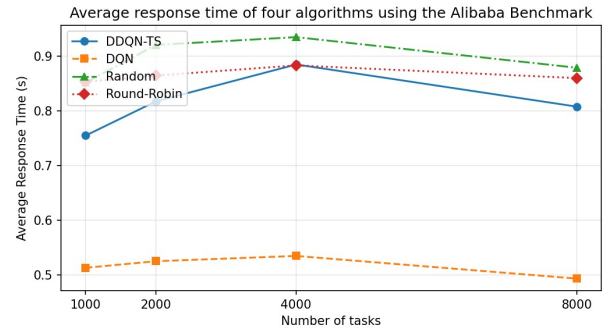


Fig. 30. art\_alibaba

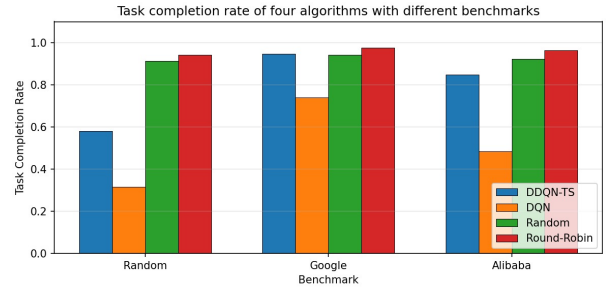


Fig. 31. tcr\_all\_benchmarks

## B. Baseline Deep Reinforcement Learning Scheduler

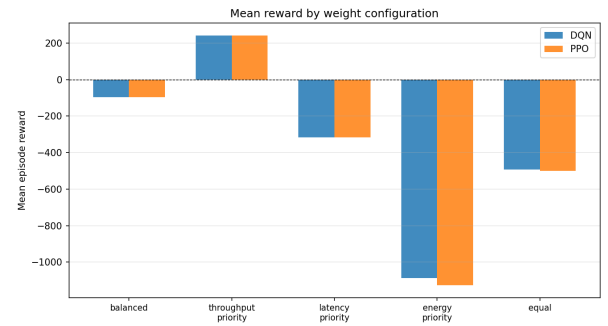


Fig. 32. sensitivity\_mean\_reward

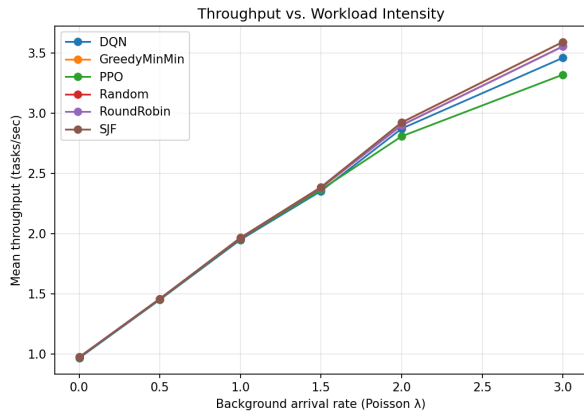


Fig. 33. throughput\_vs\_arrival

#### D. Experiment2:IncreasedServerScale(Eight-ServerEnvironment)

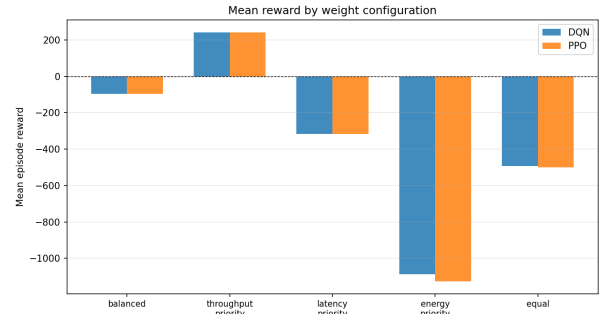


Fig. 36. sensitivity\_mean\_reward

#### C. Experiment1:HeavyWorkloadStressTest

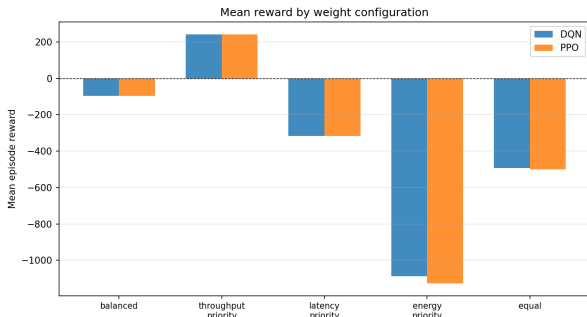


Fig. 34. sensitivity\_mean\_reward

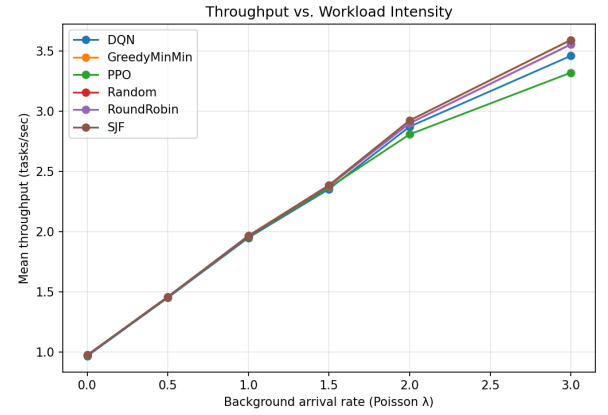


Fig. 37. throughput\_vs\_arrival

#### E. Experiment3:HeterogeneousEnergyConsumption

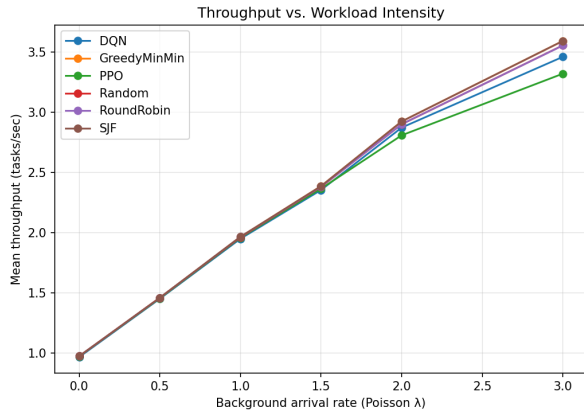


Fig. 35. throughput\_vs\_arrival

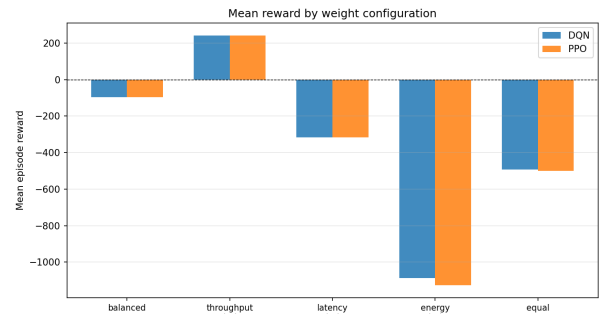


Fig. 38. sensitivity\_mean\_reward

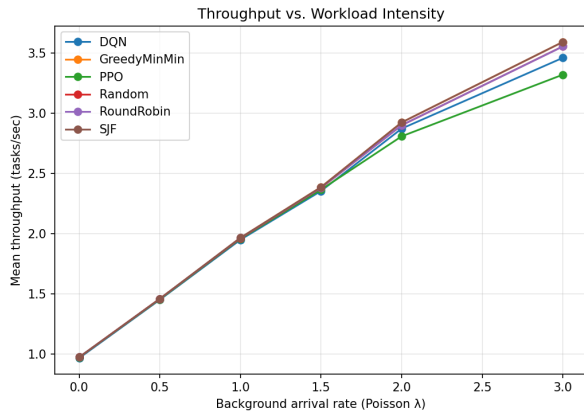


Fig. 39. throughput\_vs\_arrival

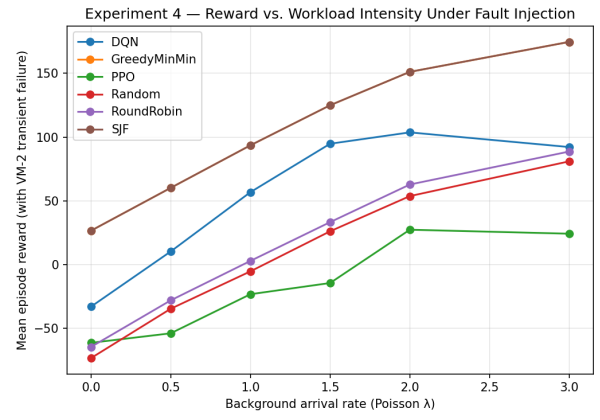


Fig. 42. resilience\_reward\_vs\_arrival

## F. Experiment4:TransientServerFailure

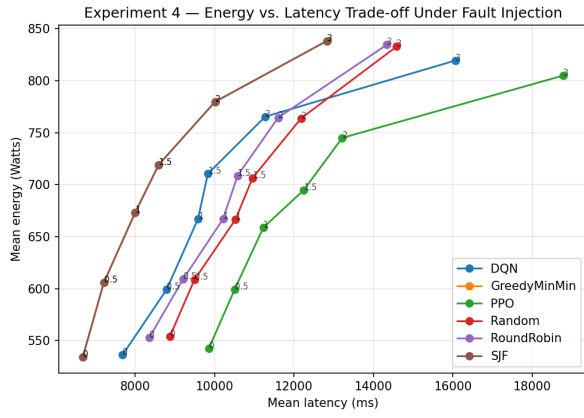


Fig. 40. resilience\_energy\_latency\_tradeoff

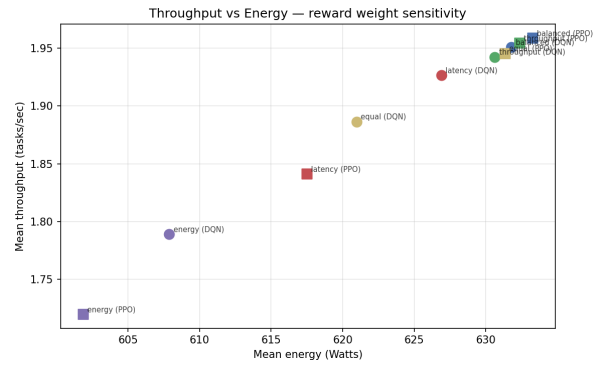


Fig. 43. sensitivity\_throughput\_vs\_energy

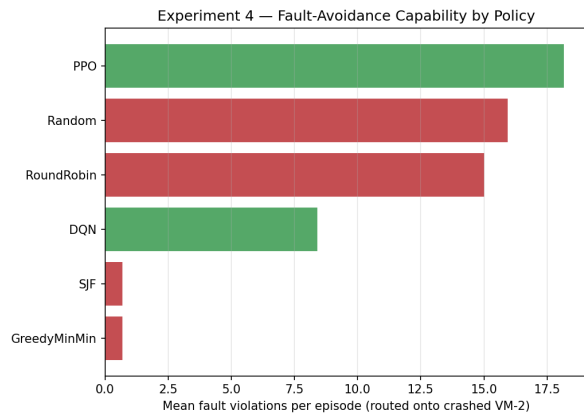


Fig. 41. resilience\_fault\_violations

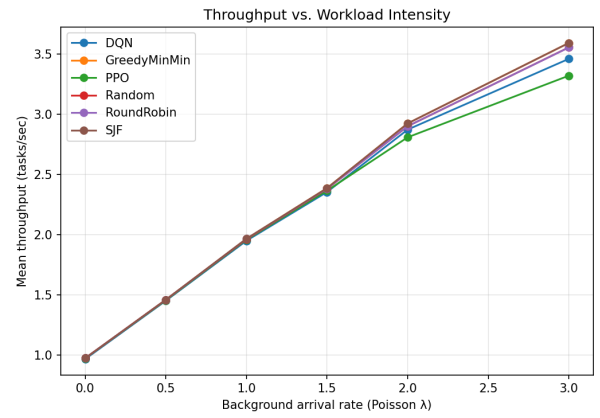


Fig. 44. throughput\_vs\_arrival